

Docker

Extra Class: MLOps



Nguyen-Thuan Duong – TA

Outline

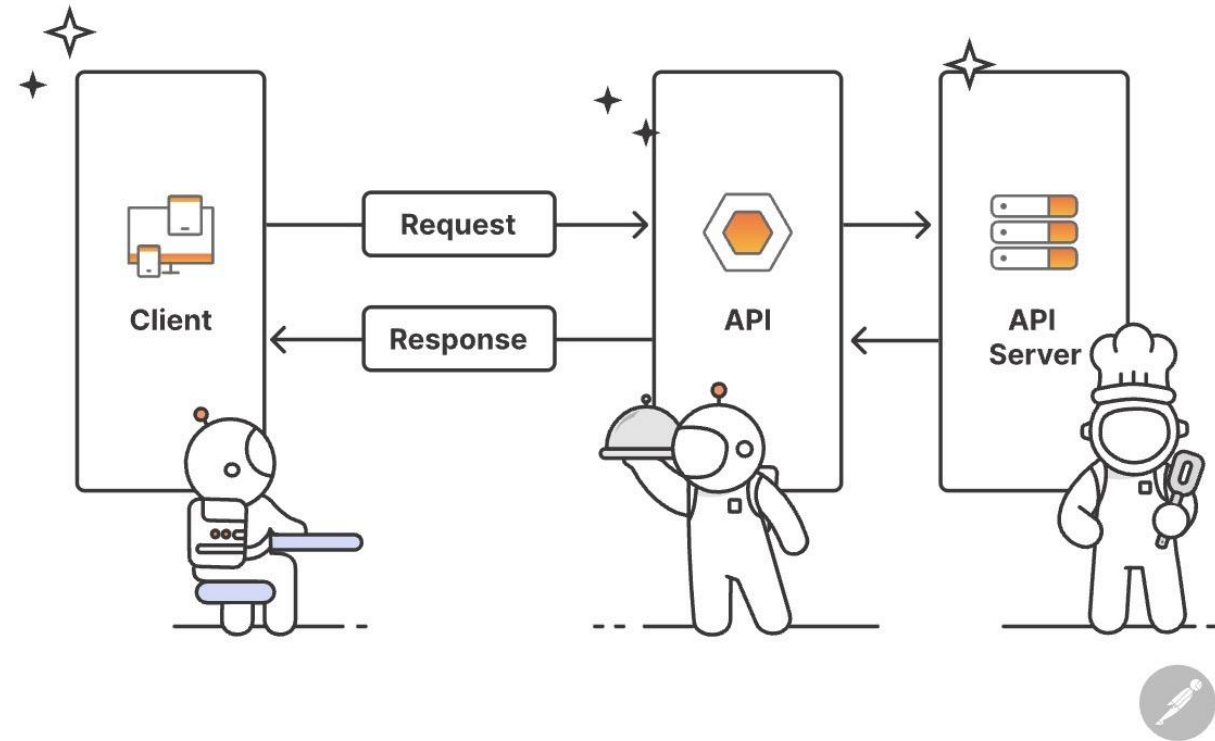
- Introduction
- Docker
- Docker Build
- Docker Run
- Docker Compose
- Docker Hub
- Question

Introduction

Introduction

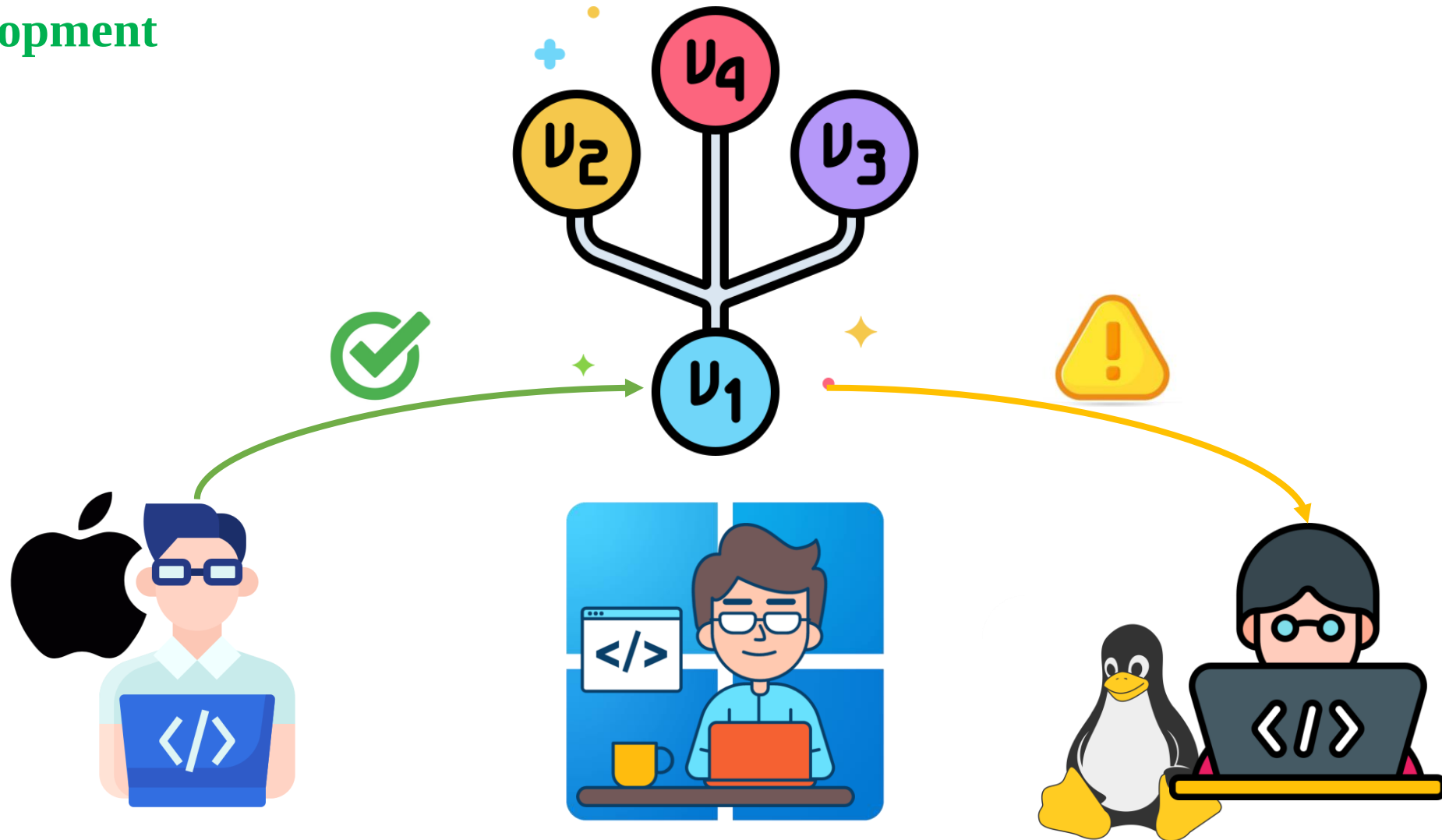
❖ Getting Started

- Frontend (User Interface);
 - HTML, CSS, JS
 - React, Flutter, ...
 - Streamlit UI, Gradio UI, ...
- Backend:
 - Server: NodeJS, FastAPI
 - APIs: REST Full, GraphQL
 - Database: SQL, NoSQL
- Infrastructure:
 - Hosting: GC, AWS
 - Networking
 - Containers
 - Monitoring



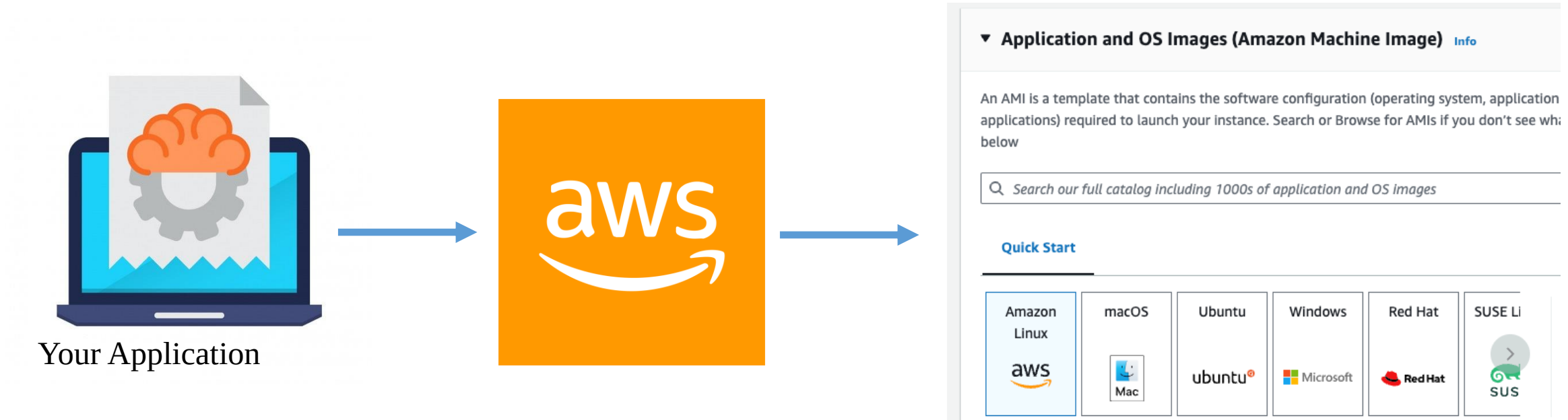
Introduction

❖ Development



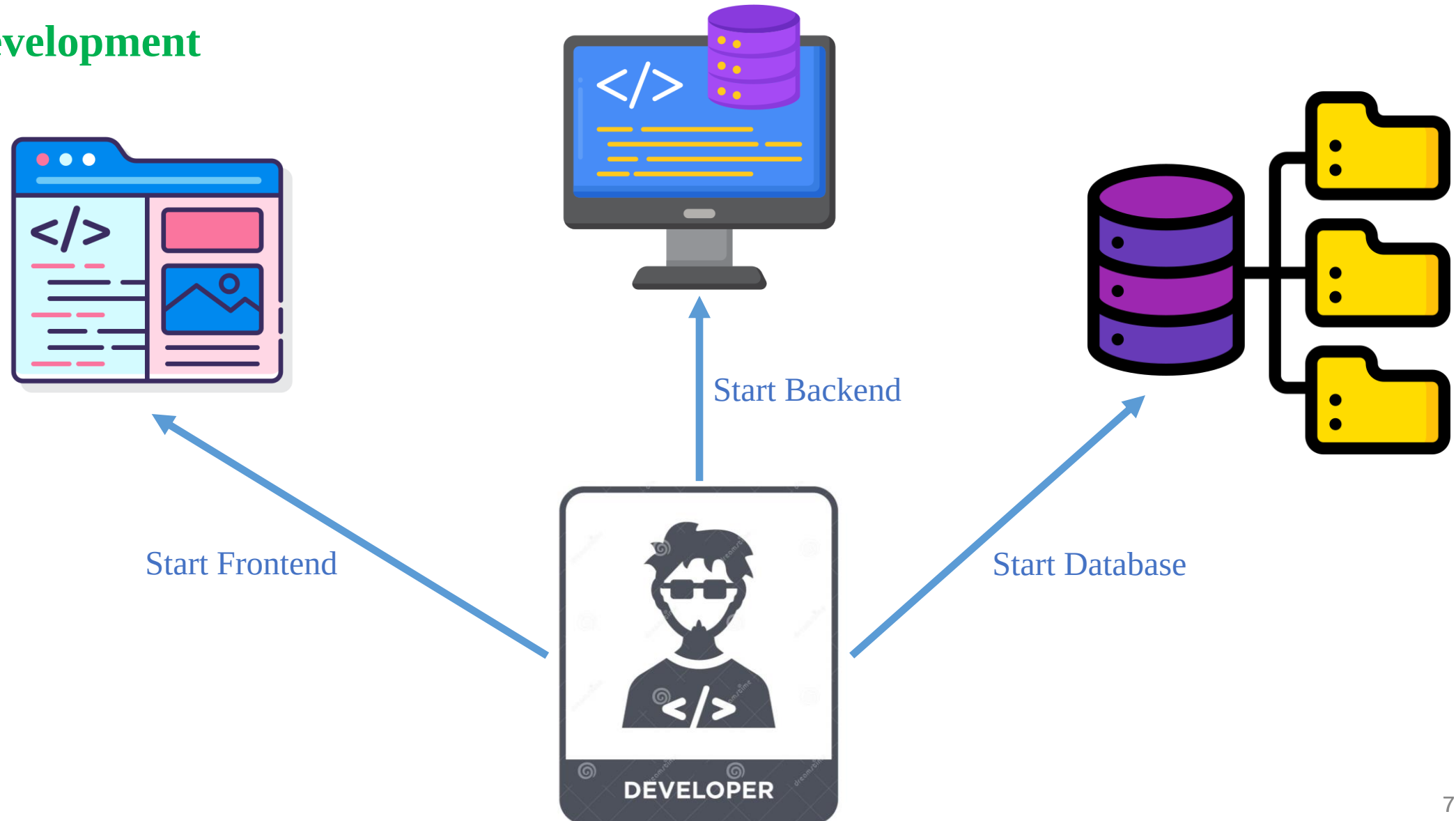
Introduction

❖ Deployment



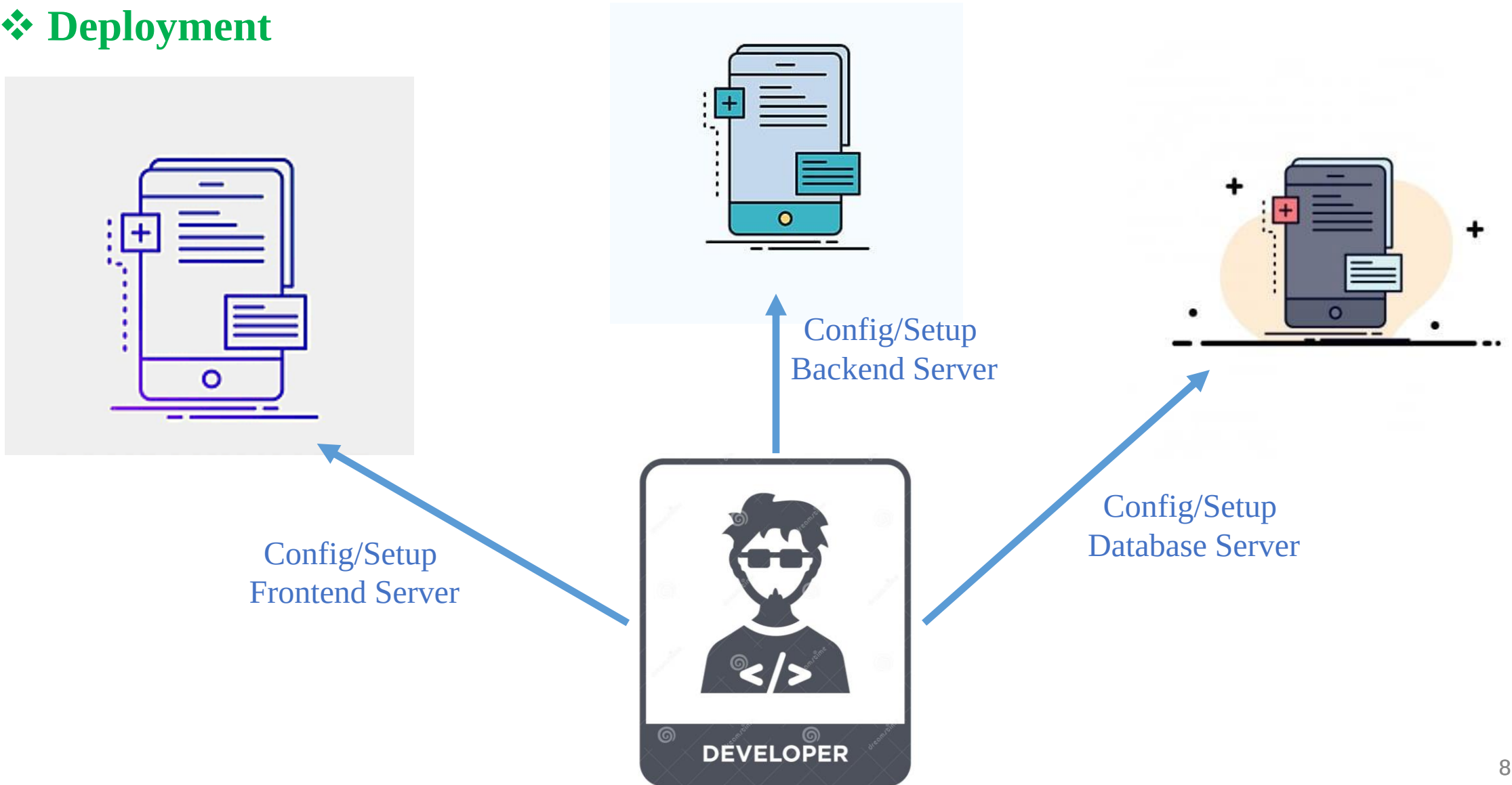
Introduction

❖ Development



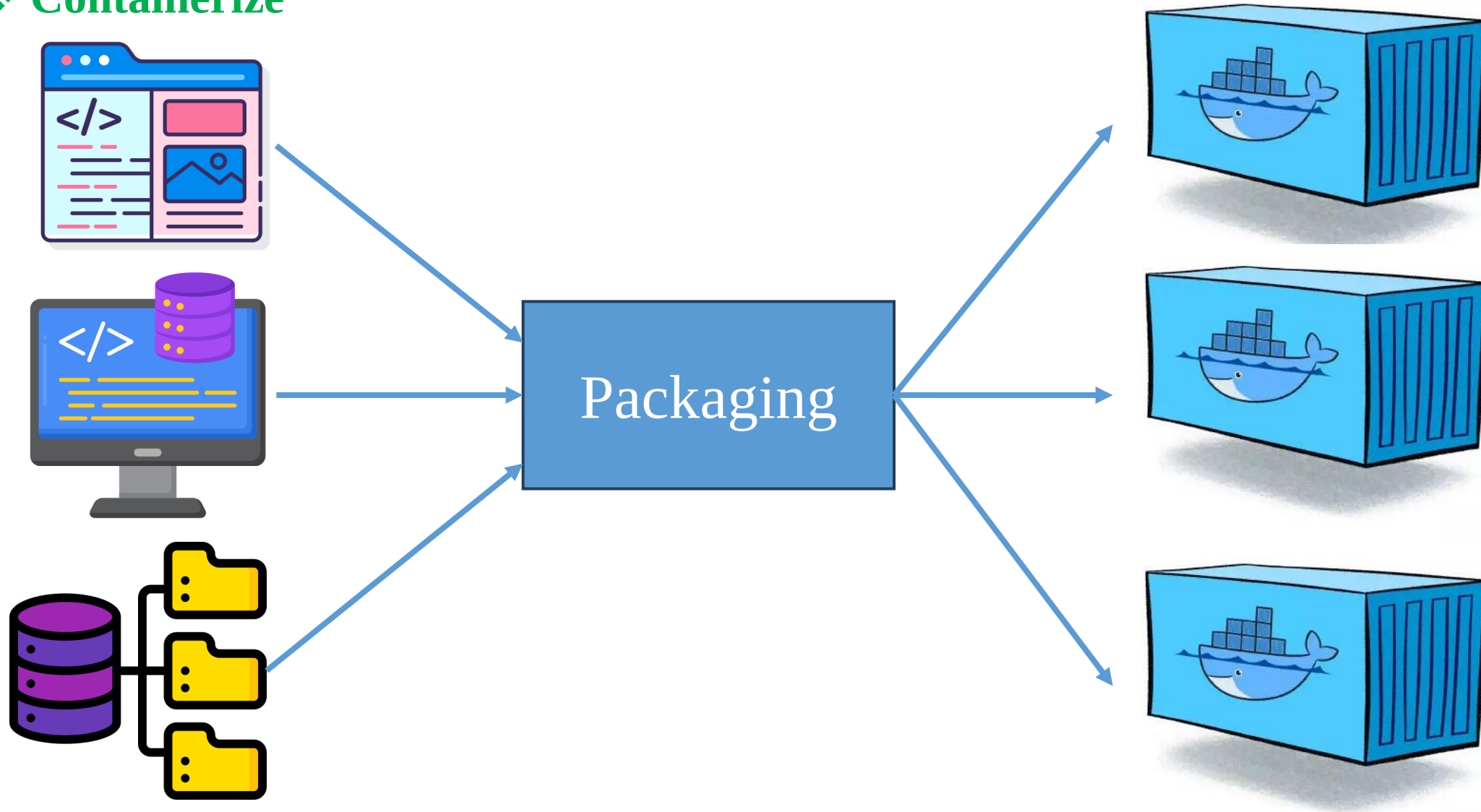
Introduction

❖ Deployment



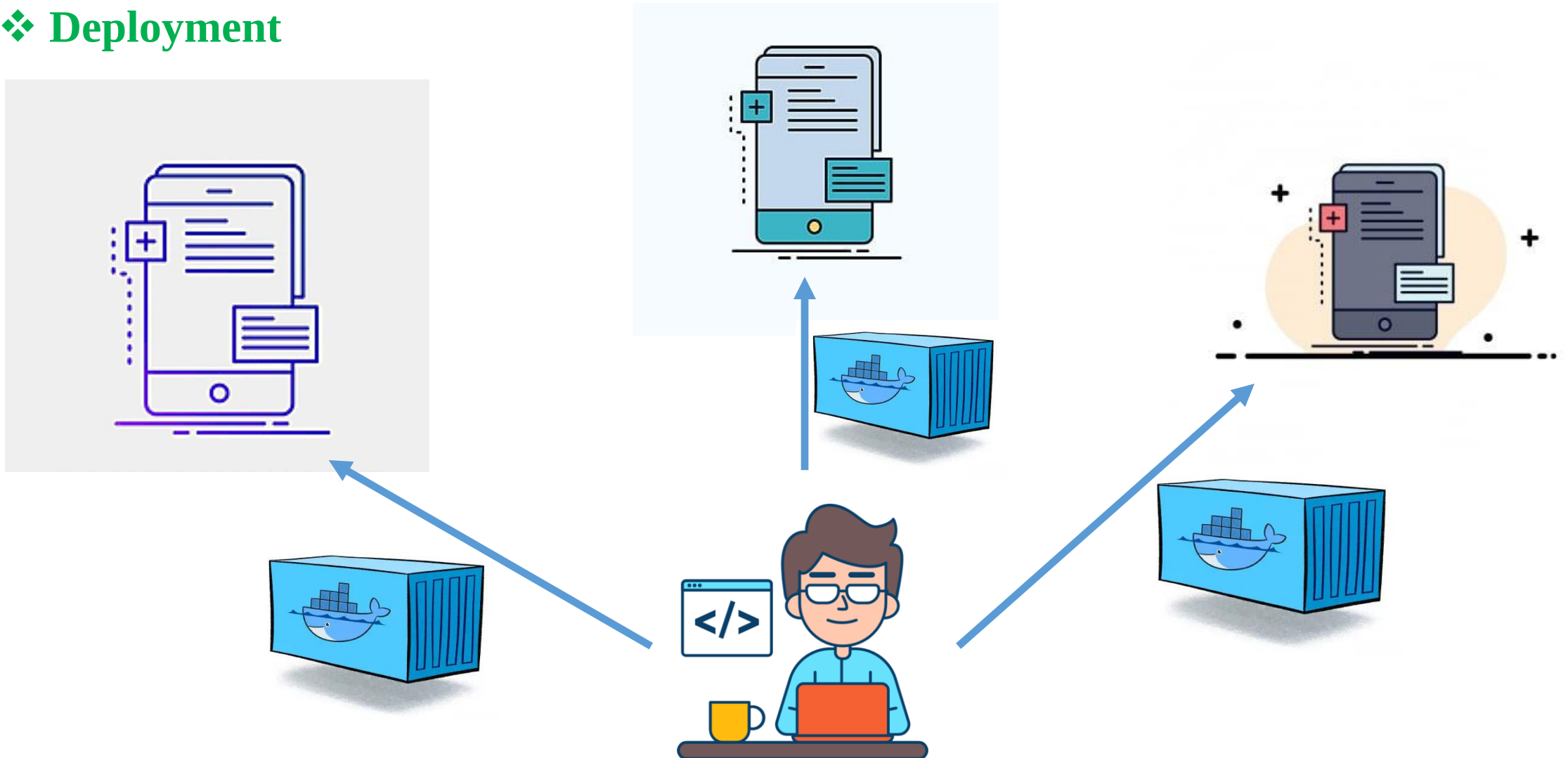
Introduction

❖ Containerize



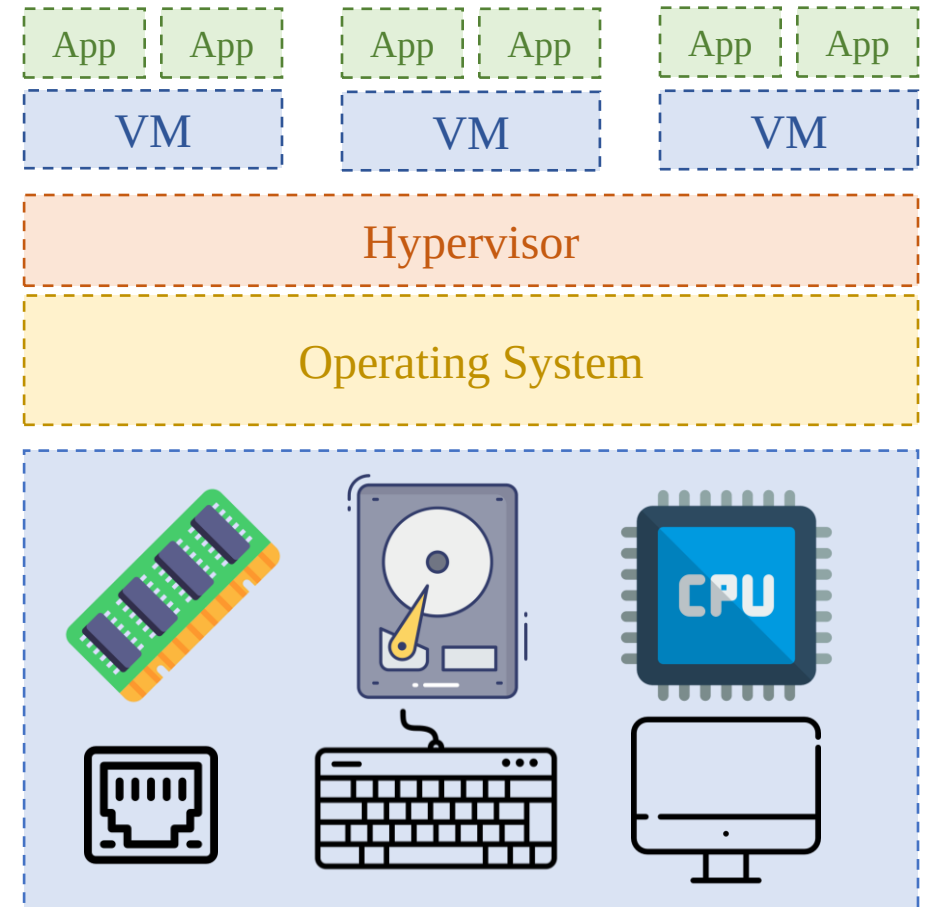
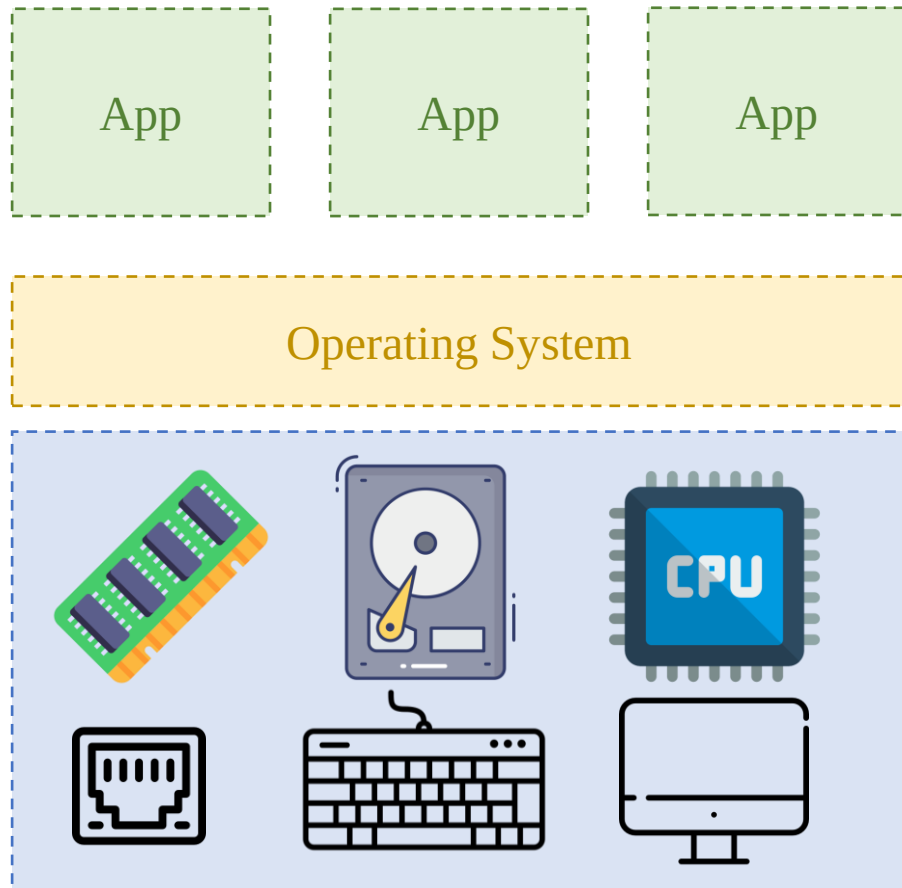
Introduction

❖ Deployment



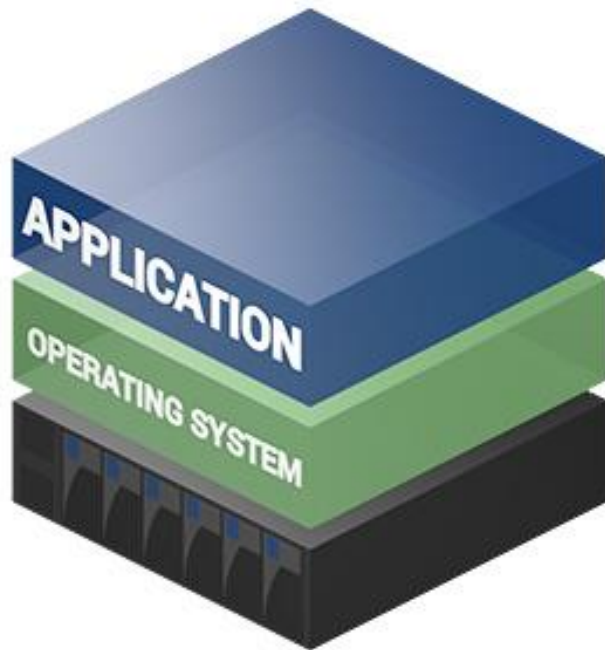
Introduction

❖ Previous Solution



Introduction

❖ Previous Solution



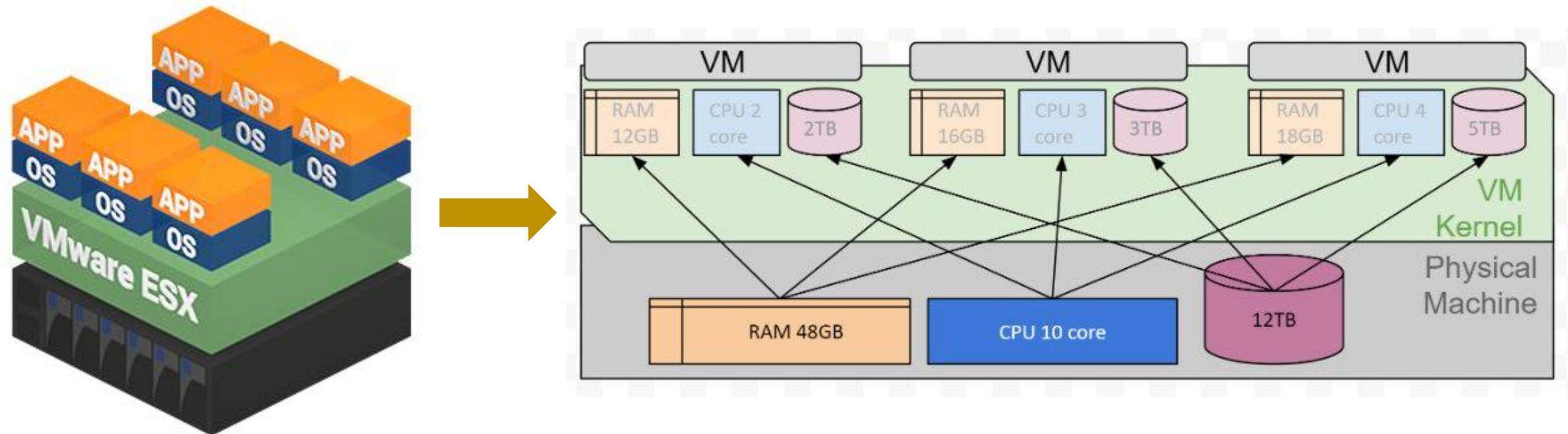
Traditional Architecture



Virtual Architecture

Introduction

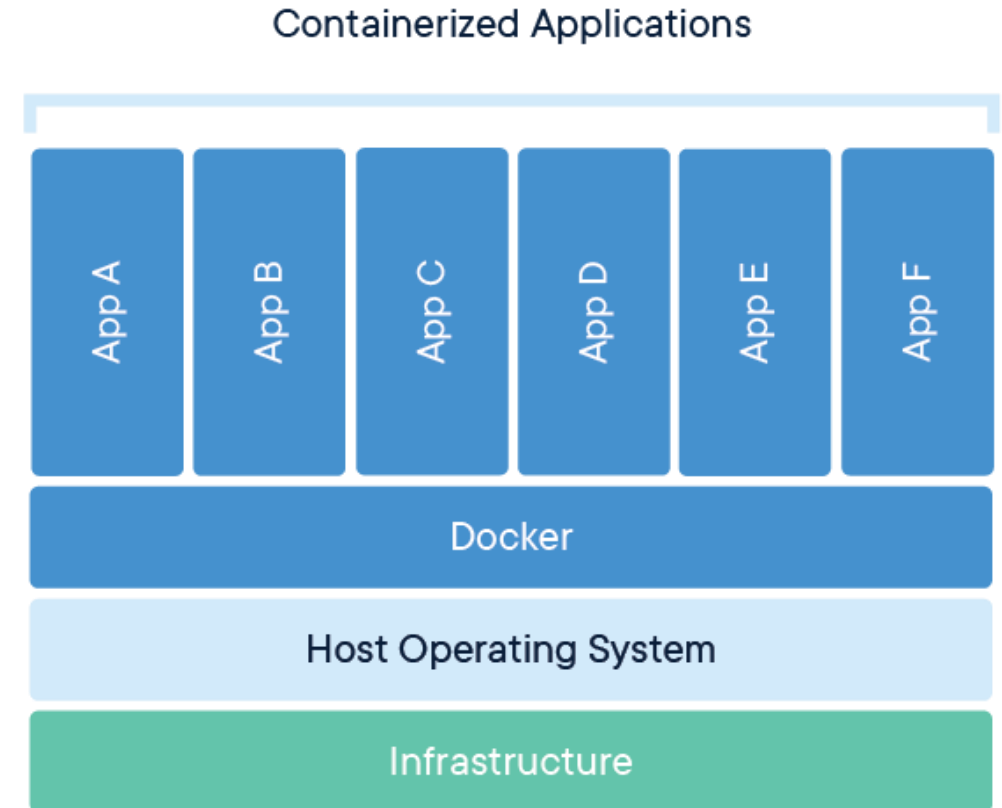
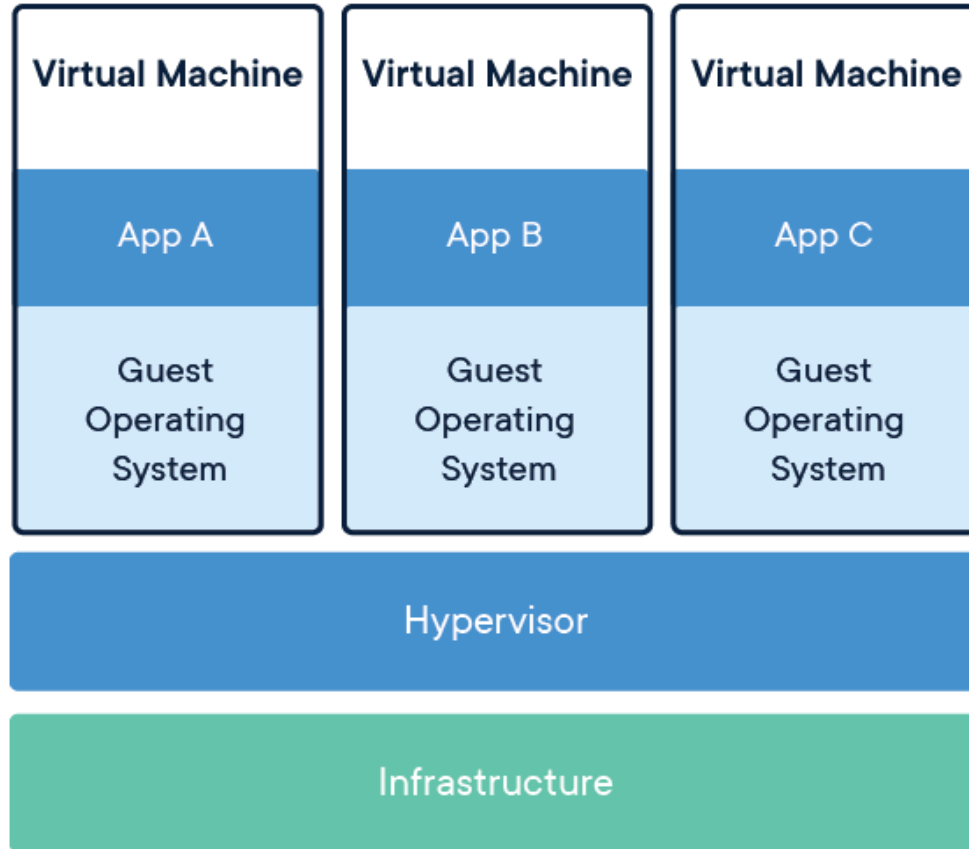
❖ Previous Solution



Allocate resources when initializing VM

Introduction

❖ Container Method

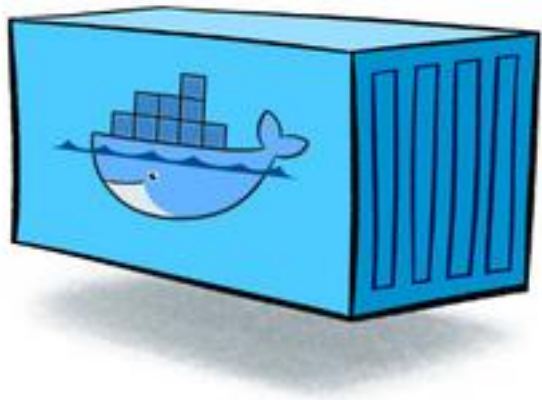


Docker

Docker

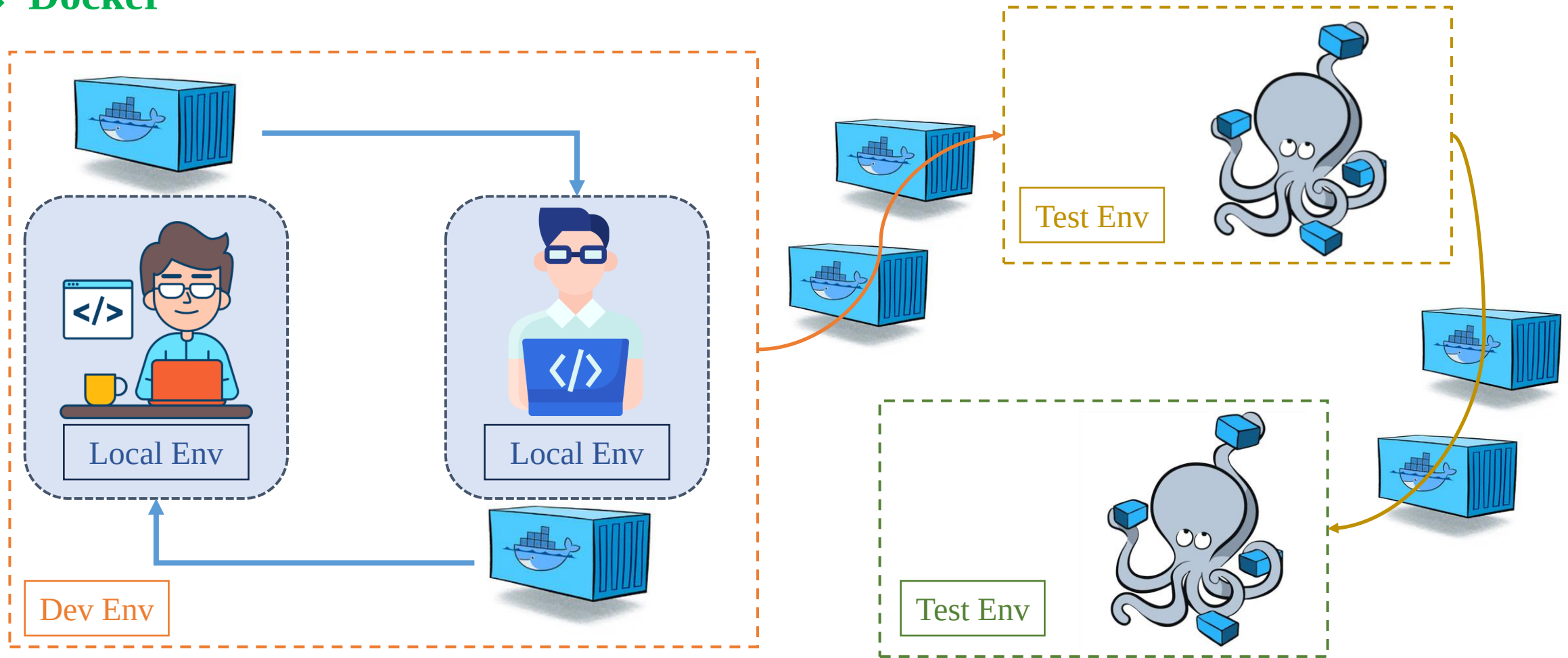
❖ Docker

- Docker is an open platform for developing, shipping, and running applications.
- Significantly reduce the delay between writing code and running it in production.



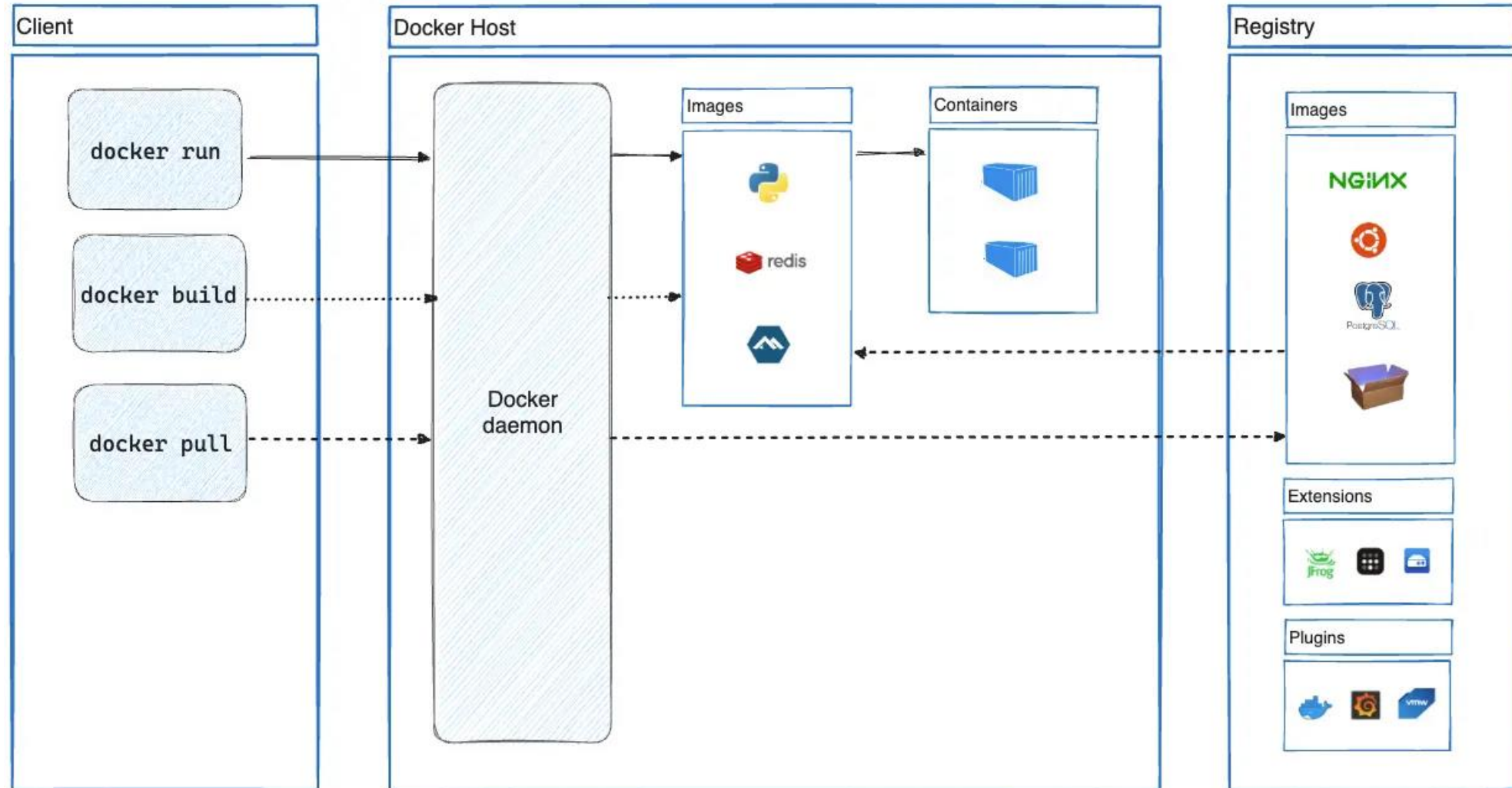
Docker

❖ Docker



Docker

❖ Docker Architecture



❖ Installation

Docker Desktop is a one-click-install application for your Mac, Linux, or Windows environment that lets you build, share, and run containerized applications

Docker CLI

Docker Engine

GUI

[Home](#) / [Manuals](#) / [Docker Desktop](#) / [Install](#) / Windows

Install Docker Desktop on Windows

Docker Desktop terms

Commercial use of Docker Desktop in larger enterprises (more than 250 employees OR annual revenue) requires a [paid subscription](#).

This page contains the download URL, information about system requirements, and instructions for installing Docker Desktop for Windows.

[Docker Desktop for Windows - x86_64](#)

[Docker Desktop for Windows - Arm \(Beta\)](#)

[Home](#) / [Manuals](#) / [Docker Desktop](#) / [Install](#) / Mac

Install Docker Desktop on Mac

Docker Desktop terms

Commercial use of Docker Desktop in larger enterprises (more than 250 employees OR annual revenue) requires a [paid subscription](#).

This page contains download URLs, information about system requirements, and instructions for installing Docker Desktop for Mac.

[Docker Desktop for Mac with Apple silicon](#)

[Docker Desktop for Mac with Intel chip](#)

[Home](#) / [Manuals](#) / [Docker Desktop](#) / [Install](#) / Linux

Install Docker Desktop on Linux

Docker Desktop terms

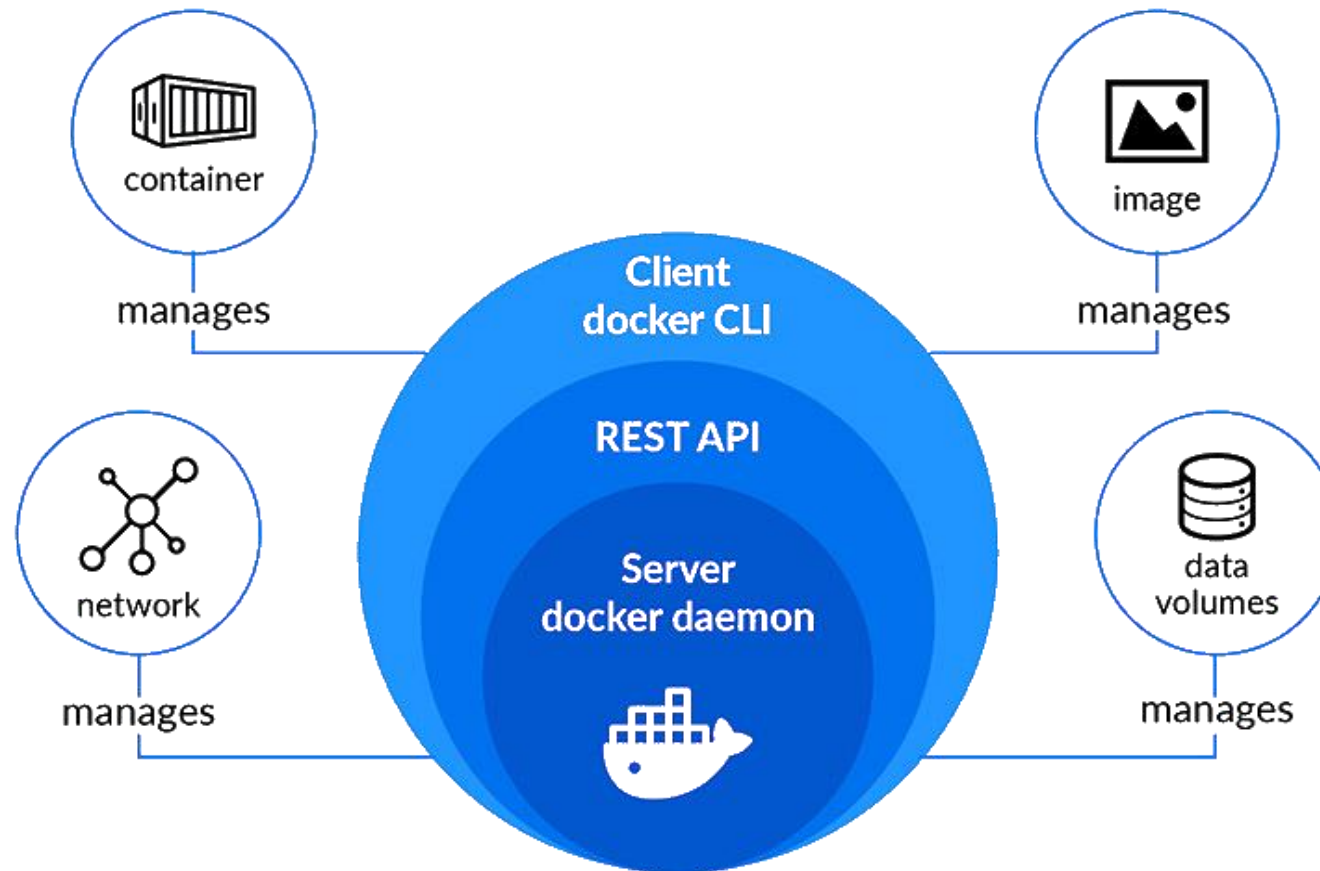
Commercial use of Docker Desktop in larger enterprises (more than 250 employees OR annual revenue) requires a [paid subscription](#).

This page contains information about general system requirements, supported platforms, and instructions for installing Docker Desktop for Linux.

Docker

❖ Deamon

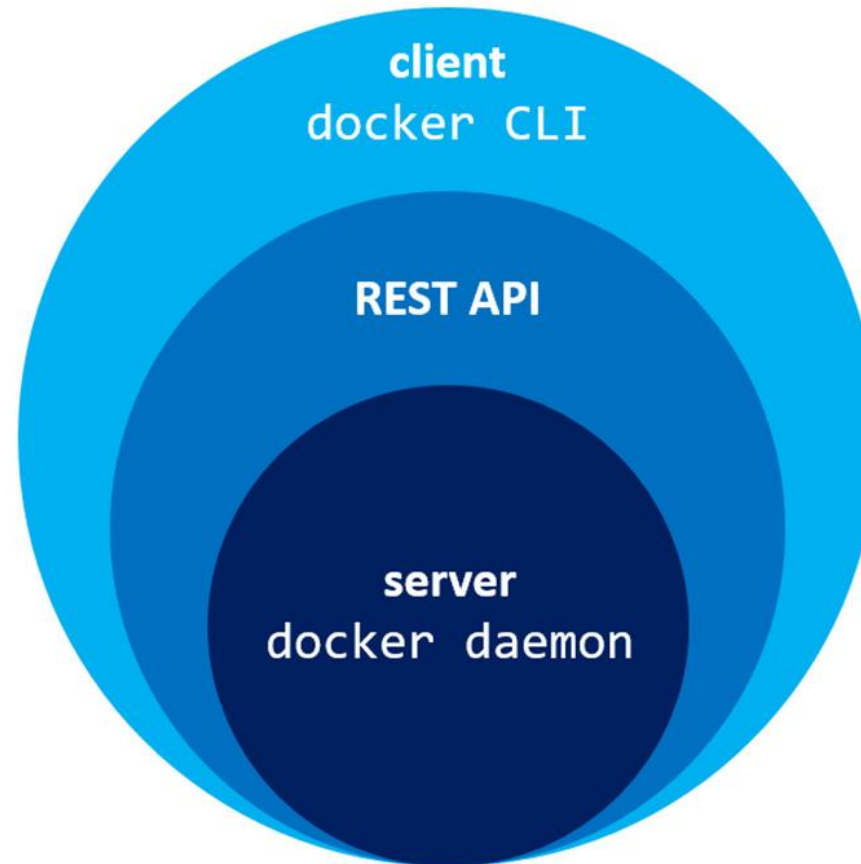
- Listen Docker API requests
- Manages Docker objects such as images, containers, networks, and volumes



Docker

❖ Engine

- Docker Engine is an open source containerization technology.
- Docker Engine acts as a client-server application.



Docker Engine

❖ Install Docker Engine

Home / Manuals / Docker Engine / Install

Install Docker Engine

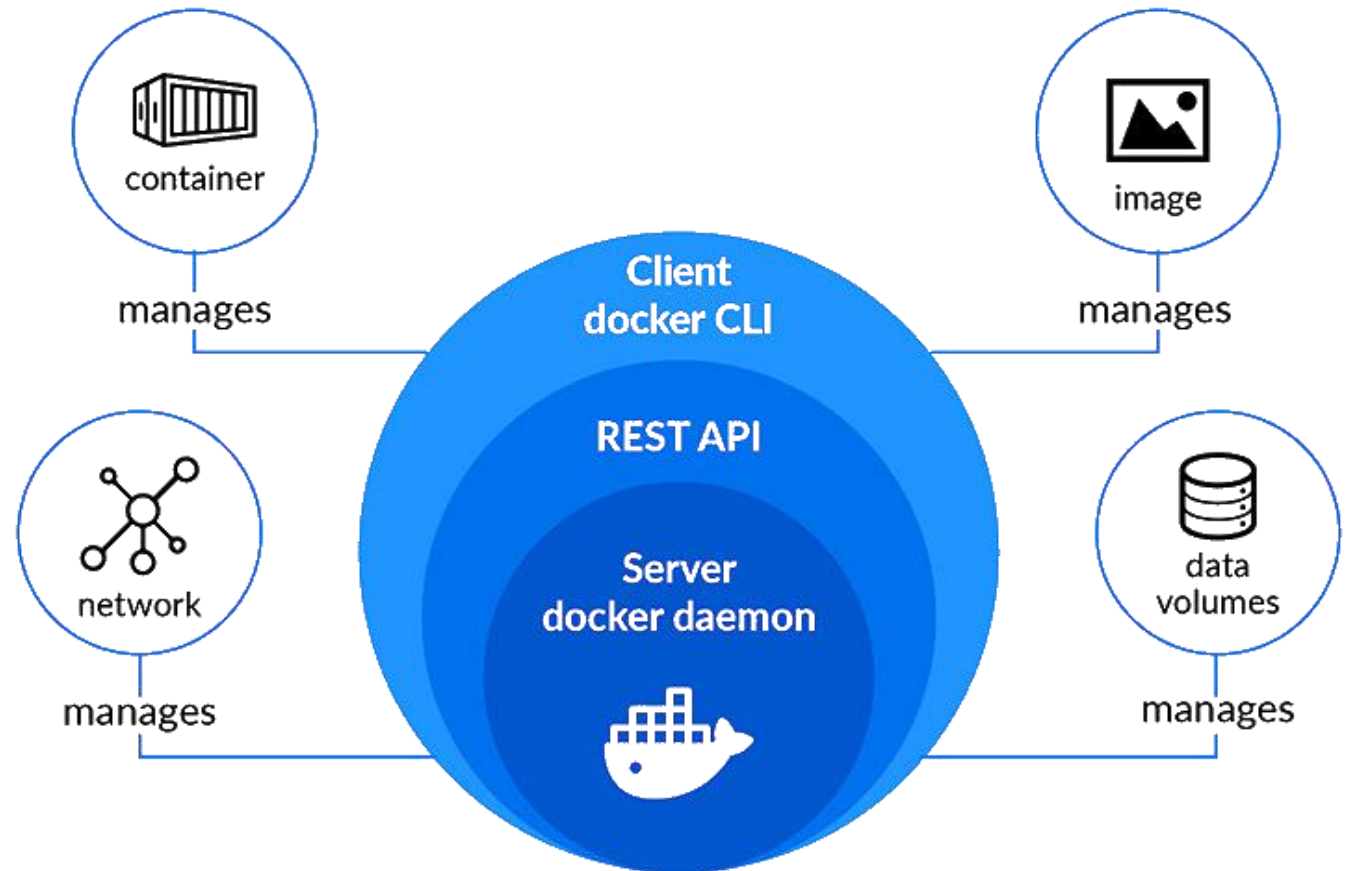
This section describes how to install Docker Engine on Linux, also known as Docker CE. Docker Engine is also available on Windows, macOS, and Linux, through Docker Desktop. For instructions on how to install Docker Desktop, see:

- [Docker Desktop for Linux](#)
- [Docker Desktop for Mac \(macOS\)](#)
- [Docker Desktop for Windows](#)

Supported platforms

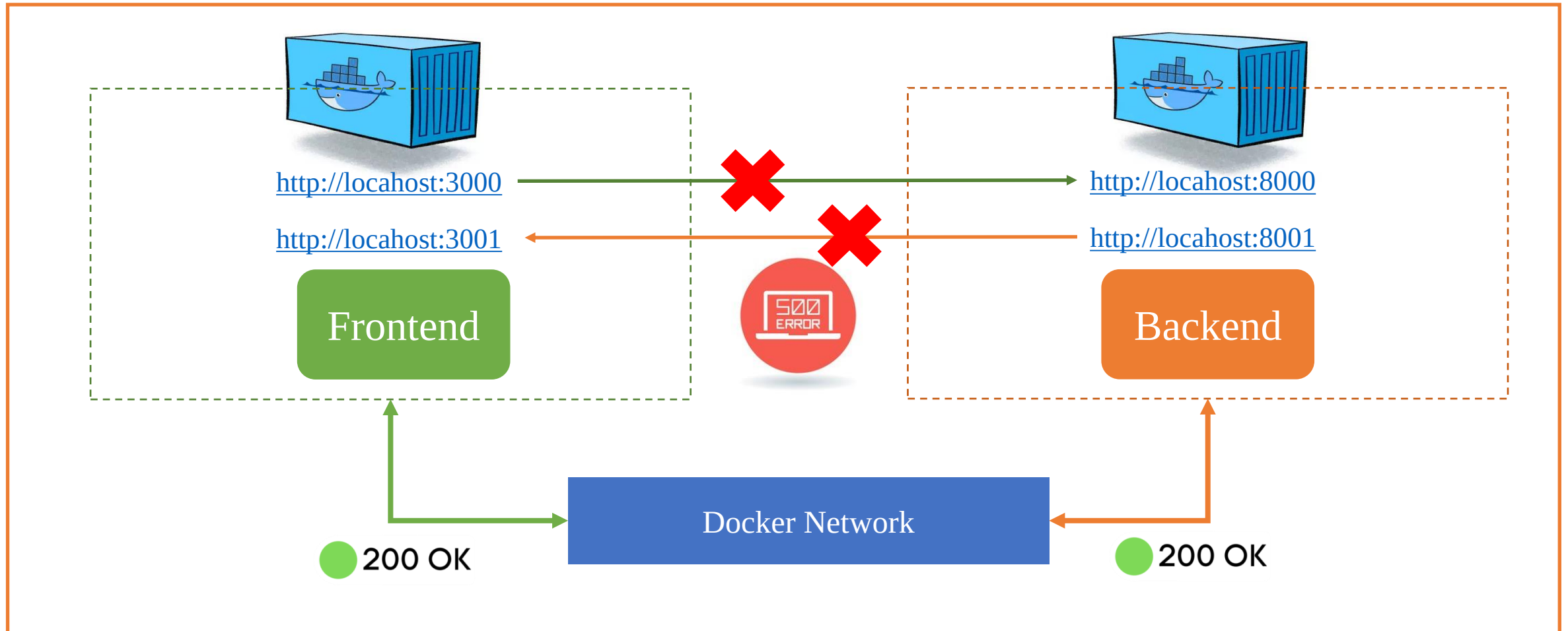
Platform	x86_64 / amd64	arm64 / aarch64	arm (32-bit)	ppc64le	s390x
CentOS	✓	✓		✓	
Debian	✓	✓	✓	✓	
Fedora	✓	✓		✓	
Raspberry Pi OS (32-bit)			✓		
RHEL	⚠	⚠			✓
SLES					✓
Ubuntu	✓	✓	✓	✓	✓
Binaries	✓	✓	✓		

⚠ = Experimental



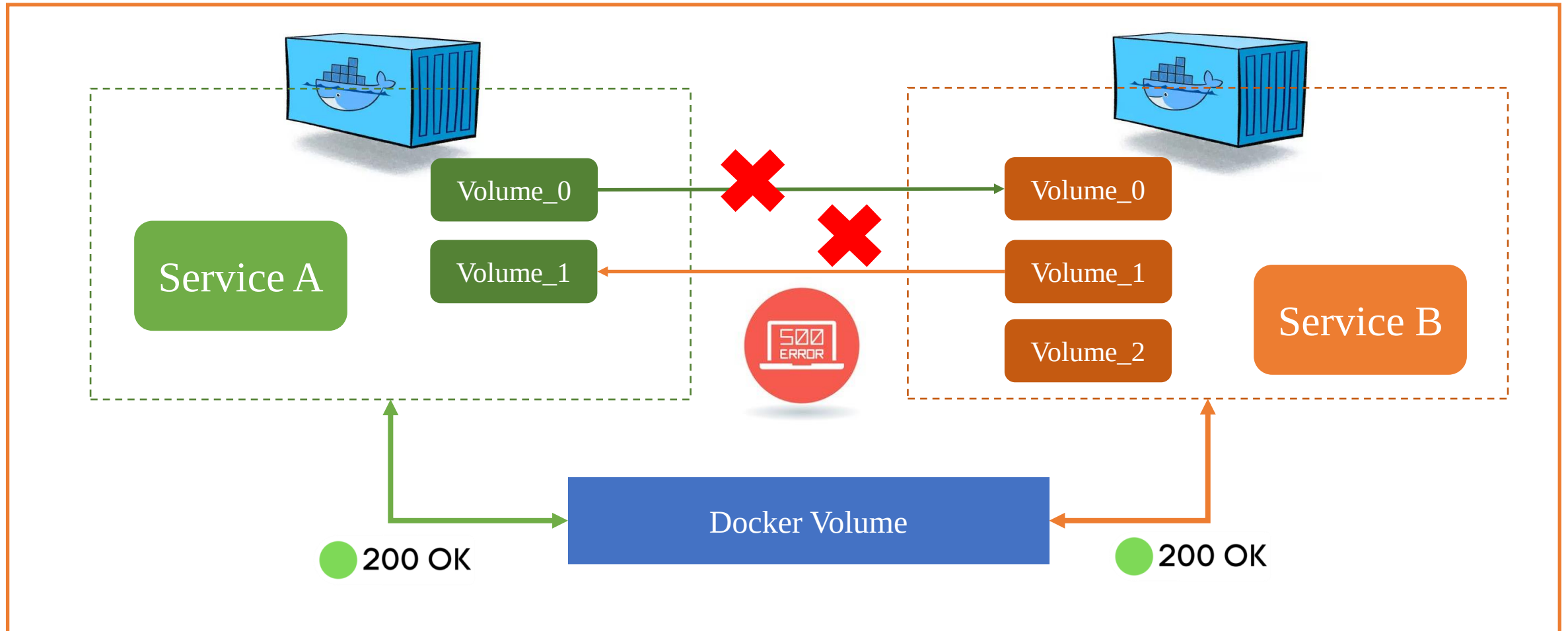
Docker Engine

❖ Networking



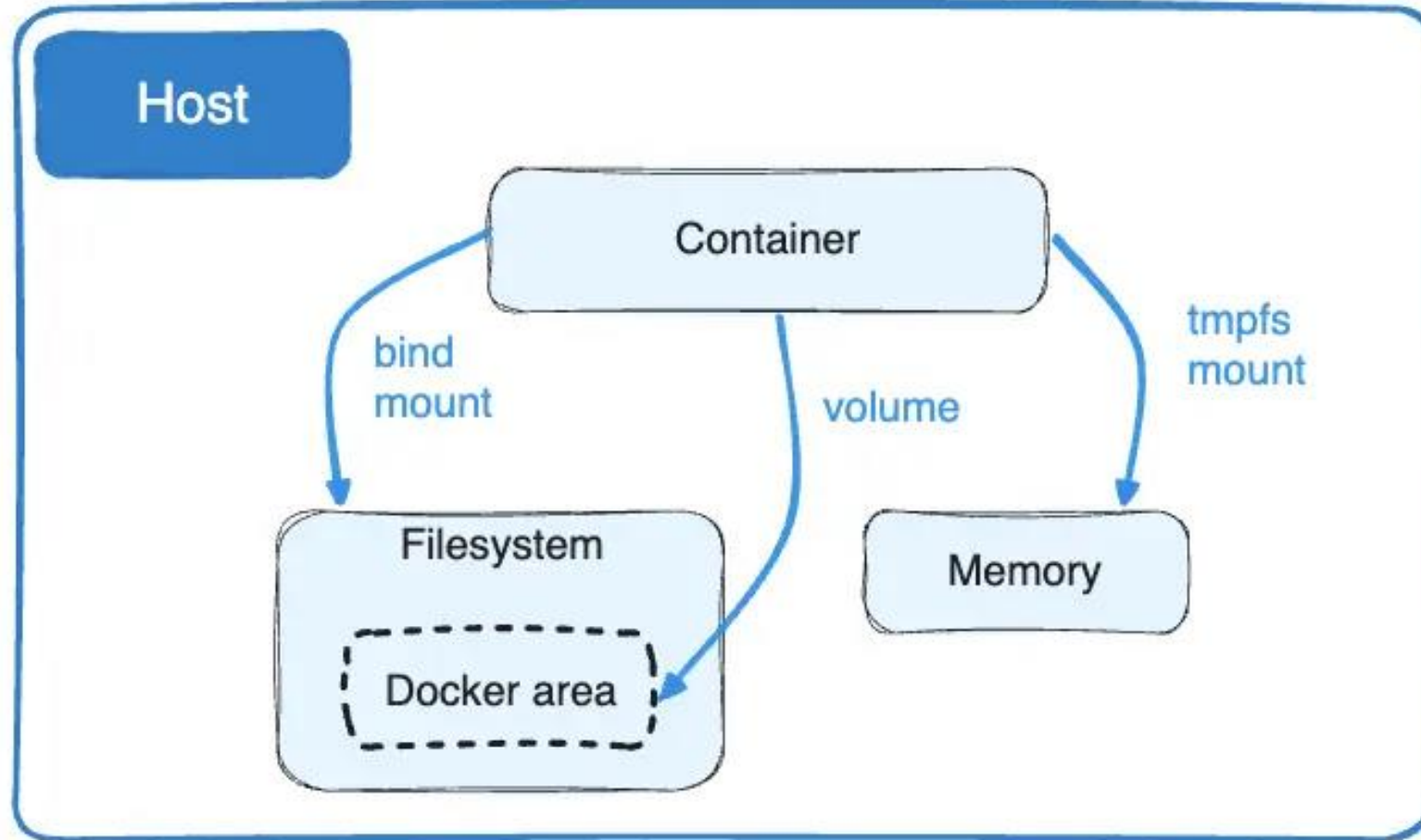
Docker Engine

❖ Storage



Docker Engine

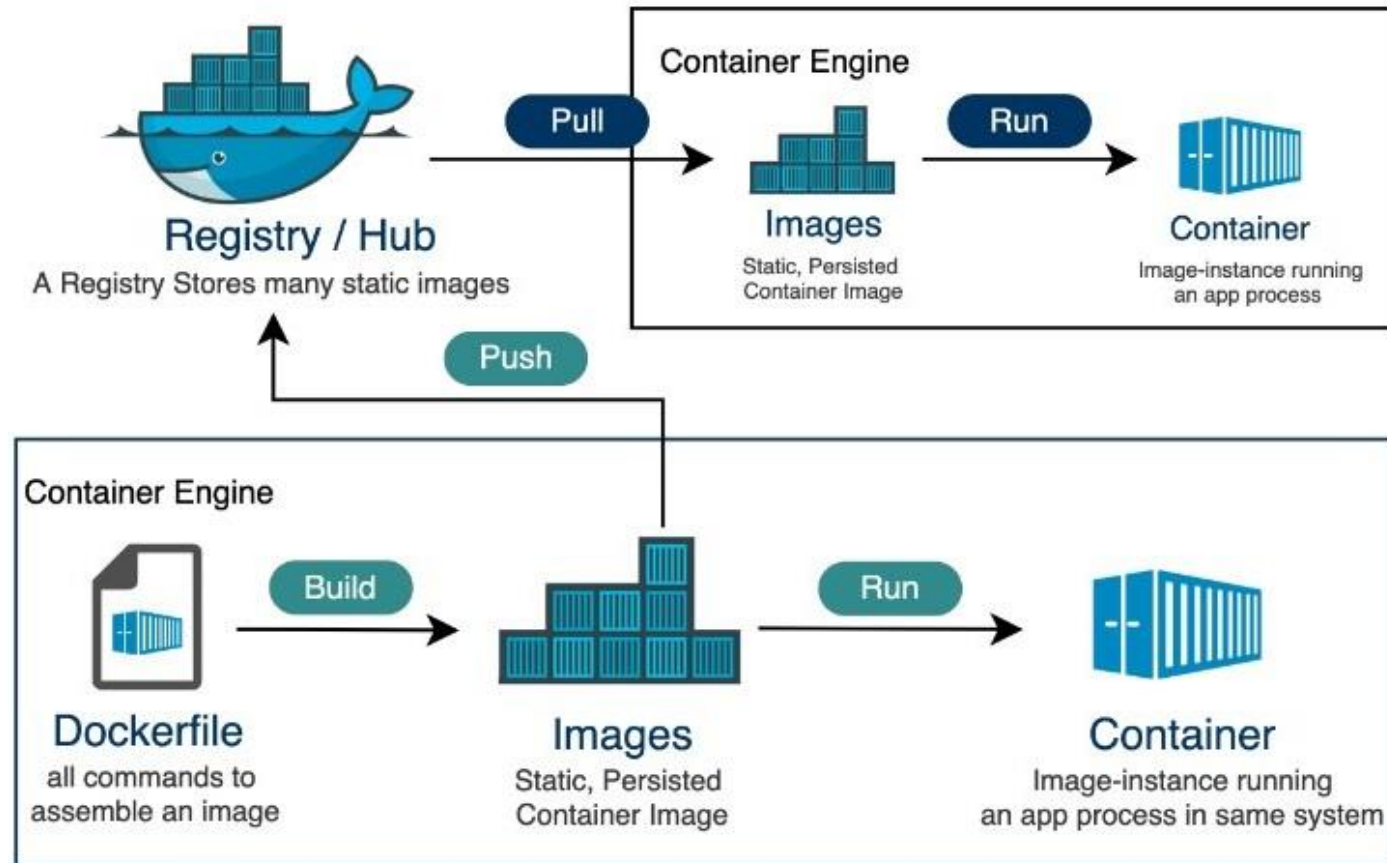
❖ Volumes



Docker

❖ Registry

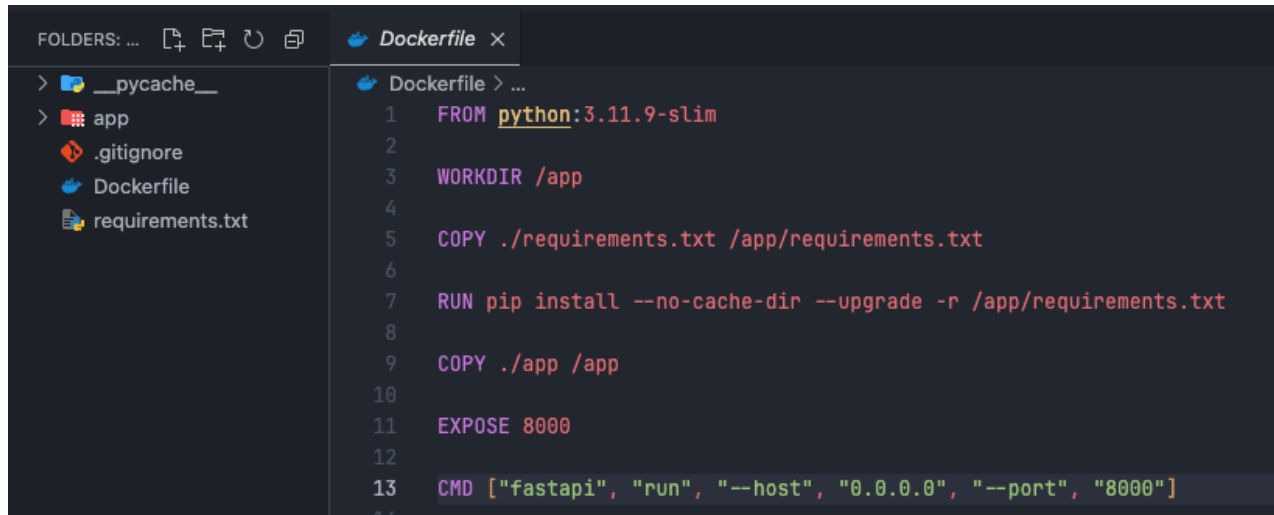
- A Docker registry stores Docker images.
- Docker Hub is a public registry.



Docker

❖ Dockerfile

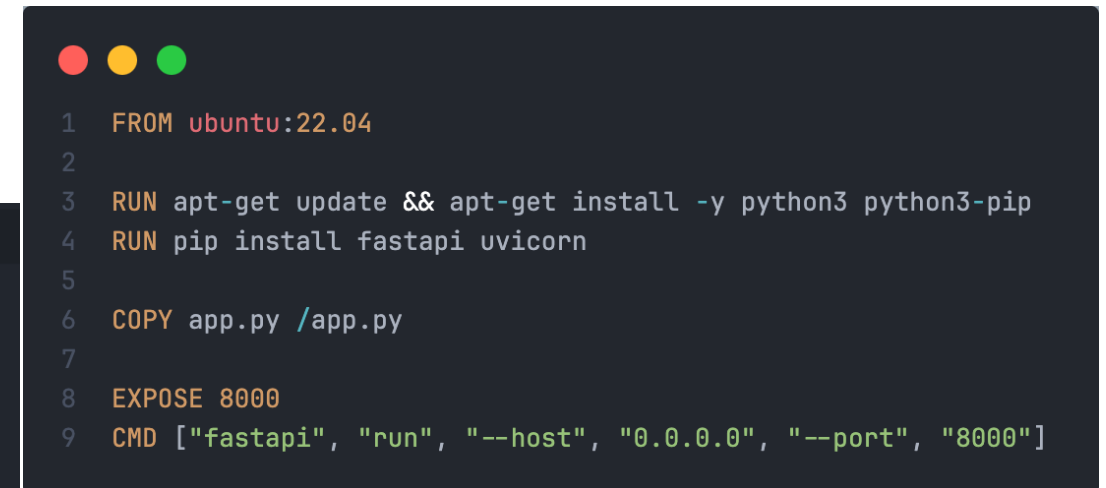
- A Dockerfile is an instruction to build a Docker image.
- A Dockerfile is a text file and has instruction syntax defined by the specification



A screenshot of a code editor showing a Dockerfile. The left sidebar shows a file tree with folders like __pycache__ and app, and files like .gitignore, Dockerfile, and requirements.txt. The main editor area shows the Dockerfile content with line numbers 1 through 13. The instructions are: FROM python:3.11.9-slim, WORKDIR /app, COPY ./requirements.txt /app/requirements.txt, RUN pip install --no-cache-dir --upgrade -r /app/requirements.txt, COPY ./app /app, EXPOSE 8000, and CMD ["fastapi", "run", "--host", "0.0.0.0", "--port", "8000"].

```
1 FROM python:3.11.9-slim
2
3 WORKDIR /app
4
5 COPY ./requirements.txt /app/requirements.txt
6
7 RUN pip install --no-cache-dir --upgrade -r /app/requirements.txt
8
9 COPY ./app /app
10
11 EXPOSE 8000
12
13 CMD ["fastapi", "run", "--host", "0.0.0.0", "--port", "8000"]
```

Python Image



A screenshot of a code editor showing a Dockerfile. The left sidebar shows a file tree with folders like __pycache__ and app, and files like .gitignore, Dockerfile, and requirements.txt. The main editor area shows the Dockerfile content with line numbers 1 through 9. The instructions are: FROM ubuntu:22.04, RUN apt-get update && apt-get install -y python3 python3-pip, RUN pip install fastapi uvicorn, COPY app.py /app.py, EXPOSE 8000, and CMD ["fastapi", "run", "--host", "0.0.0.0", "--port", "8000"].

```
1 FROM ubuntu:22.04
2
3 RUN apt-get update && apt-get install -y python3 python3-pip
4 RUN pip install fastapi uvicorn
5
6 COPY app.py /app.py
7
8 EXPOSE 8000
9 CMD ["fastapi", "run", "--host", "0.0.0.0", "--port", "8000"]
```

Ubuntu Image

Docker

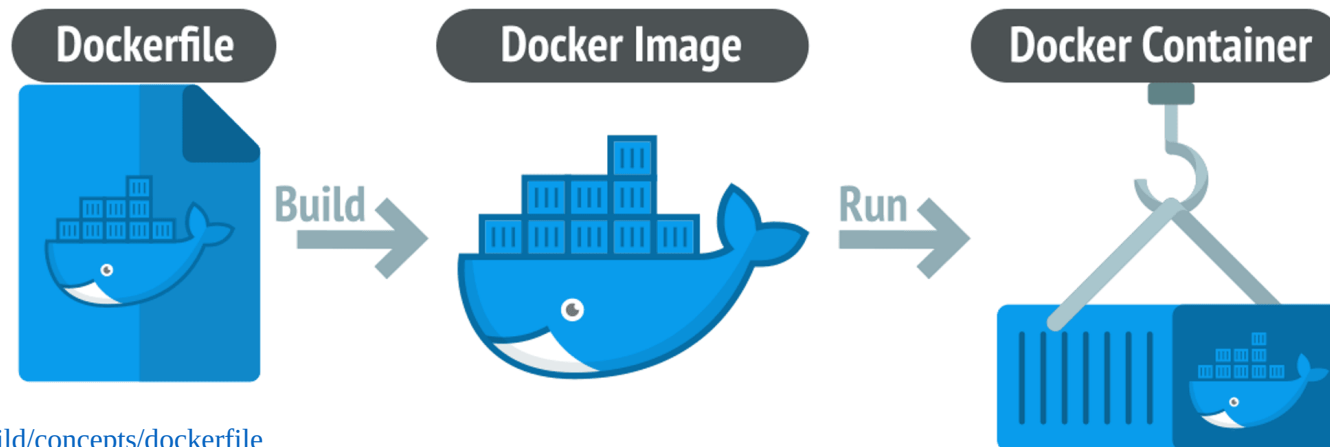
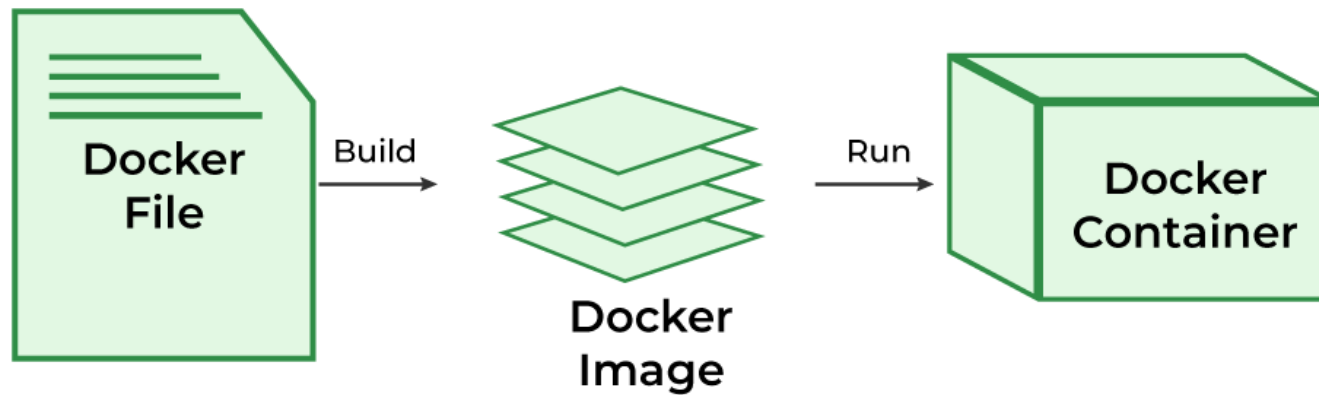
❖ Instructions

FROM	The base image, which defines the starting point for your build.
RUN	Executes a shell command in Ubuntu
WORKDIR	Set the working directory for any instructions: RUN, CMD, ...
COPY	Copy the file from local into the root directory of image
EXPOSE	Instruction informs Docker that the container listens on the specified network ports at runtime
CMD	Set the command that is run when start the container

Docker

❖ Docker Image

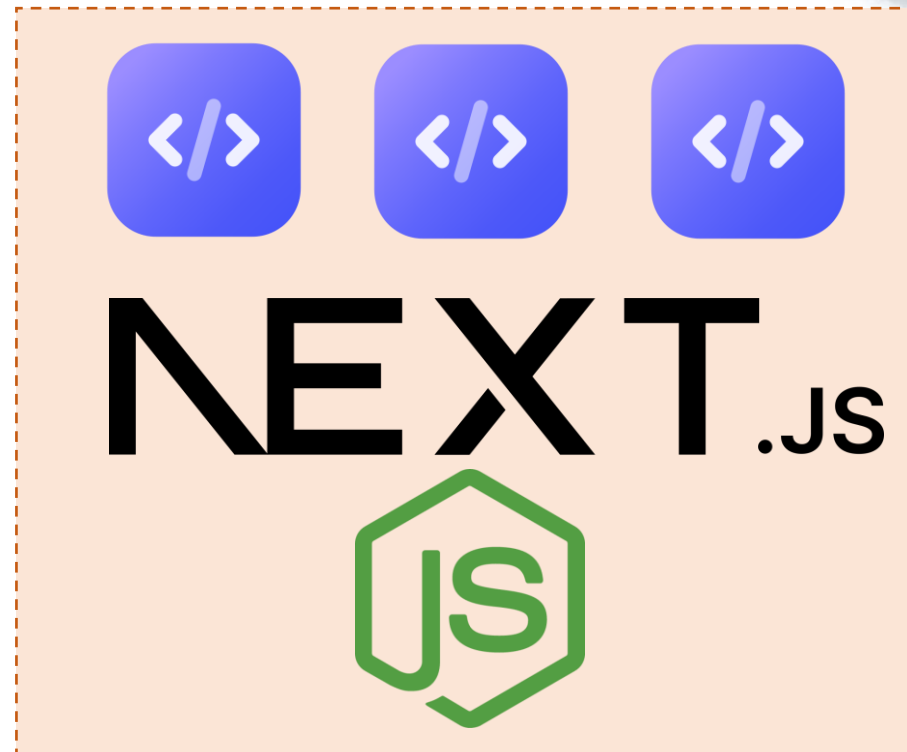
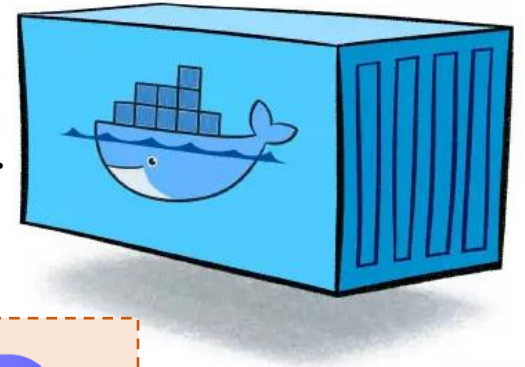
- An image is a template with instructions for creating a Docker container.
- Each instruction in Dockerfile creates a layer in image.



Docker

❖ Docker Container

- A container is a lightweight, standalone, and executable software package.
- Include everything an application needs to run: the code, runtime, libraries, and system tools.



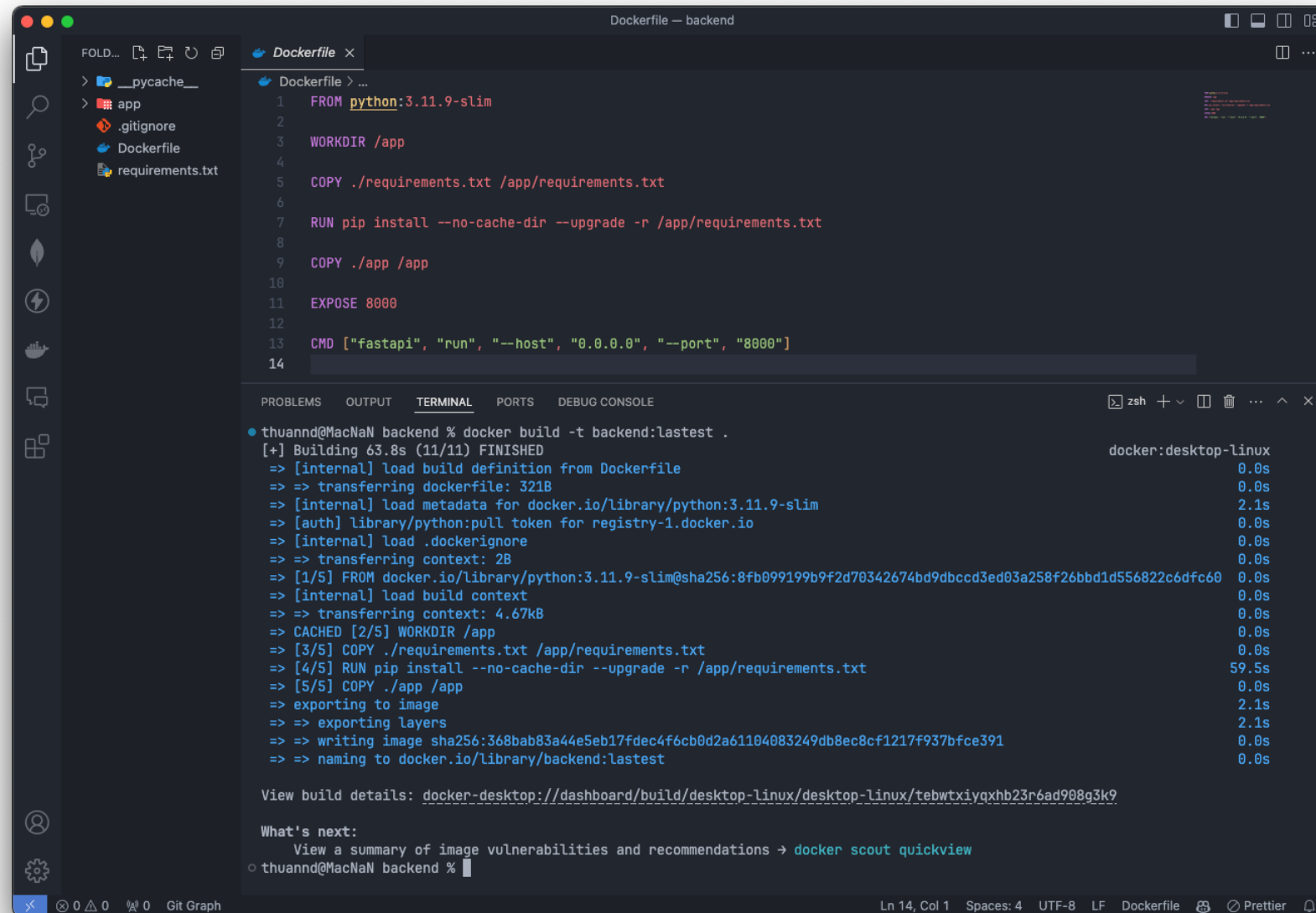
Docker Build

Docker Build

❖ Building

```
docker build -t <name>:<tag> <build context>
```

```
docker build -t backend:lastest .
```



The screenshot shows a code editor with a Dockerfile and its build output in a terminal. The Dockerfile is named 'Dockerfile' and is located in the 'backend' directory. The build output shows the command 'docker build -t backend:lastest .' being executed, and the build process is completed successfully, resulting in a new image named 'backend:lastest'.

```
Dockerfile
1 FROM python:3.11.9-slim
2
3 WORKDIR /app
4
5 COPY ./requirements.txt /app/requirements.txt
6
7 RUN pip install --no-cache-dir --upgrade -r /app/requirements.txt
8
9 COPY ./app /app
10
11 EXPOSE 8000
12
13 CMD ["fastapi", "run", "--host", "0.0.0.0", "--port", "8000"]
14
```

```
thuannd@MacNaN backend % docker build -t backend:lastest .
[+] Building 63.8s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 321B
=> [internal] load metadata for docker.io/library/python:3.11.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.11.9-slim@sha256:8fb099199b9f2d70342674bd9dbccd3ed03a258f26bbd1d556822c6dffc60
=> [internal] load build context
=> => transferring context: 4.67kB
=> CACHED [2/5] WORKDIR /app
=> [3/5] COPY ./requirements.txt /app/requirements.txt
=> [4/5] RUN pip install --no-cache-dir --upgrade -r /app/requirements.txt
=> [5/5] COPY ./app /app
=> exporting to image
=> => exporting layers
=> => writing image sha256:368bab83a44e5eb17fdec4f6cb0d2a61104083249db8ec8cf1217f937bfce391
=> => naming to docker.io/library/backend:lastest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/tebwtxiyqxhb23r6ad908g3k9

What's next:
  View a summary of image vulnerabilities and recommendations -> docker scout quickview
thuannd@MacNaN backend %
```


Docker Build

❖ Building: multi-stage

```
1 FROM python:3.11.9-slim
2
3 WORKDIR /app
4
5 COPY ./requirements.txt /app/requirements.txt
6
7 RUN pip install --no-cache-dir --upgrade -r /app/requirements.txt
8
9 COPY ./app /app
10
11 EXPOSE 8000
12
13 CMD ["fastapi", "run", "--host", "0.0.0.0", "--port", "8000"]
```

```
• thuannd@MacNaN backend % docker build -t backend:lastest .
[+] Building 63.8s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 321B
=> [internal] load metadata for docker.io/library/python:3.11.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.11.9-slim@sha256:8fb099199b9f2d70342674bd9dbd
=> [internal] load build context
=> => transferring context: 4.67kB
=> CACHED [2/5] WORKDIR /app
=> [3/5] COPY ./requirements.txt /app/requirements.txt
=> [4/5] RUN pip install --no-cache-dir --upgrade -r /app/requirements.txt
=> [5/5] COPY ./app /app
=> exporting to image
=> => exporting layers
=> => writing image sha256:368bab83a44e5eb17fdec4f6cb0d2a61104083249db8ec8cf1217f937bf
=> => naming to docker.io/library/backend:lastest
```

Docker Build

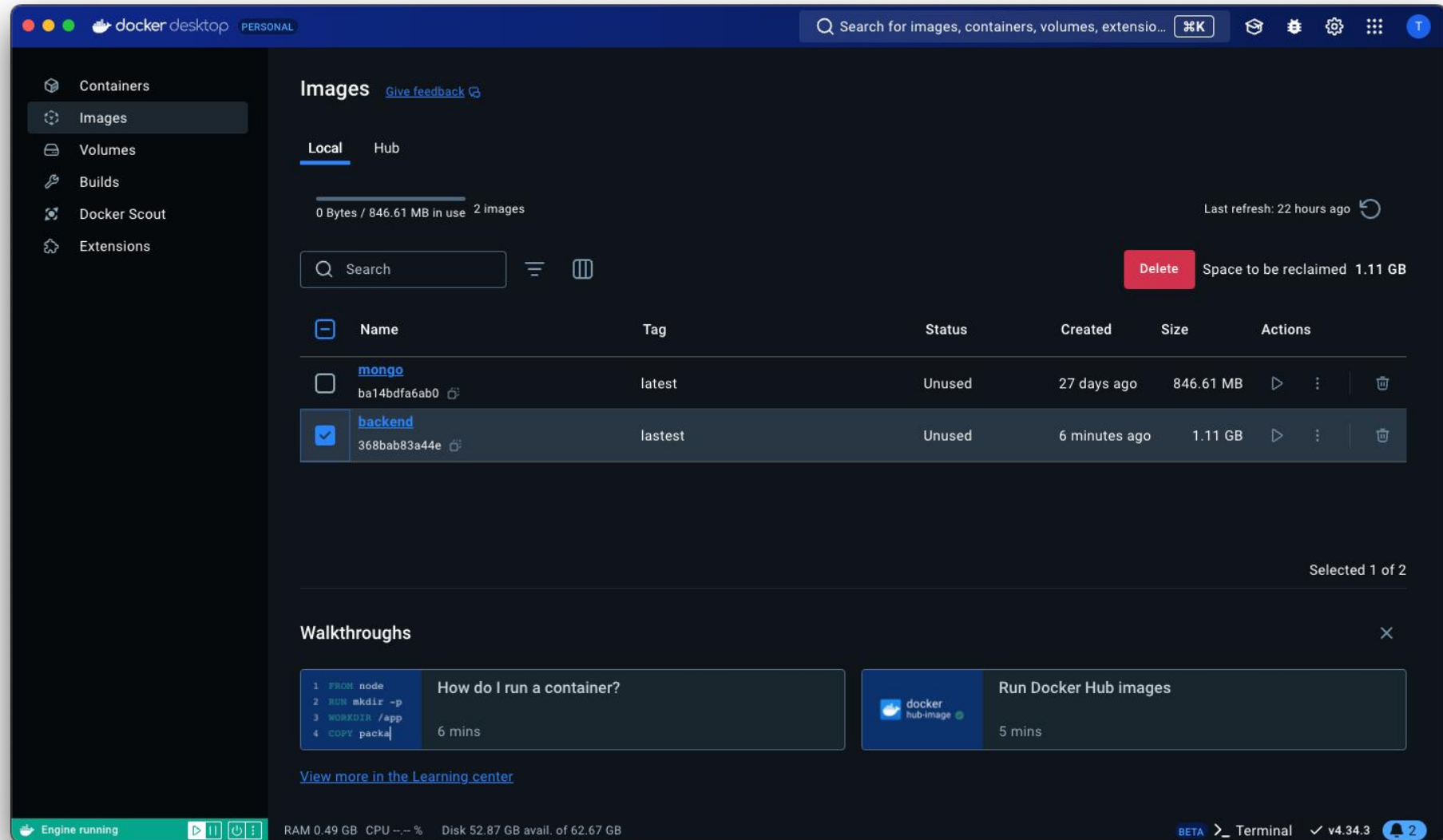
❖ Building: multi-stage

```
1 FROM python:3.11.9-slim
2
3 WORKDIR /app
4
5 COPY ./requirements.txt /app/requirements.txt
6
7 RUN pip install --no-cache-dir --upgrade -r /app/requirements.txt
8
9 COPY ./app /app
10
11 EXPOSE 8000
12
13 CMD ["fastapi", "run", "--host", "0.0.0.0", "--port", "8000"]
```

```
• thuannd@MacNaN backend % docker build -t backend:lastest .
[+] Building 63.8s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 321B
=> [internal] load metadata for docker.io/library/python:3.11.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.11.9-slim@sha256:8fb099199b9f2d70342674bd9dbc
=> [internal] load build context
=> => transferring context: 4.67kB
=> CACHED [2/5] WORKDIR /app
=> [3/5] COPY ./requirements.txt /app/requirements.txt
=> [4/5] RUN pip install --no-cache-dir --upgrade -r /app/requirements.txt
=> [5/5] COPY ./app /app
=> exporting to image
=> => exporting layers
=> => writing image sha256:368bab83a44e5eb17fdec4f6cb0d2a61104083249db8ec8cf1217f937bf
=> => naming to docker.io/library/backend:lastest
```

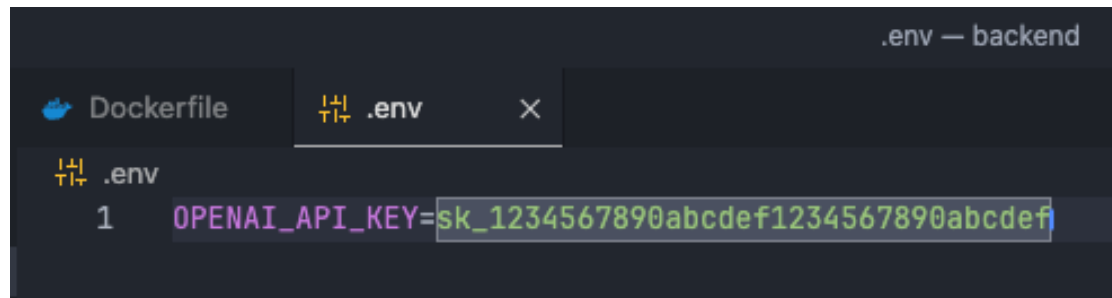
Docker Build

❖ Building

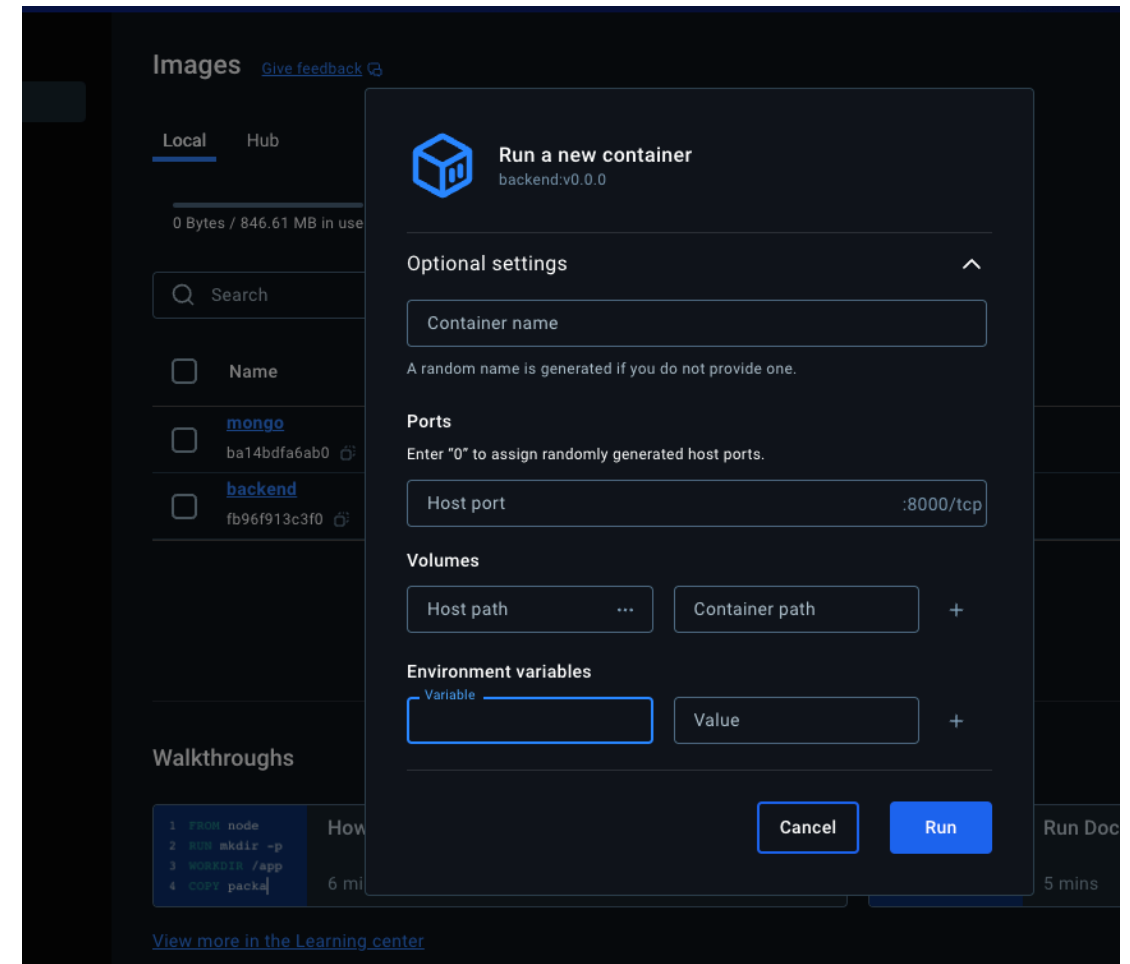


Docker Build

❖ Building: secrets



The screenshot shows a code editor with a tab labeled ".env — backend". The editor contains a single line of code: `1 OPENAI_API_KEY=sk_1234567890abcdef1234567890abcdef`. The text is highlighted in green.



```
docker build --secret OPENAI_API_KEY=sk_1234567890abcdef1234567890abcdef -t backend:lastest .
```

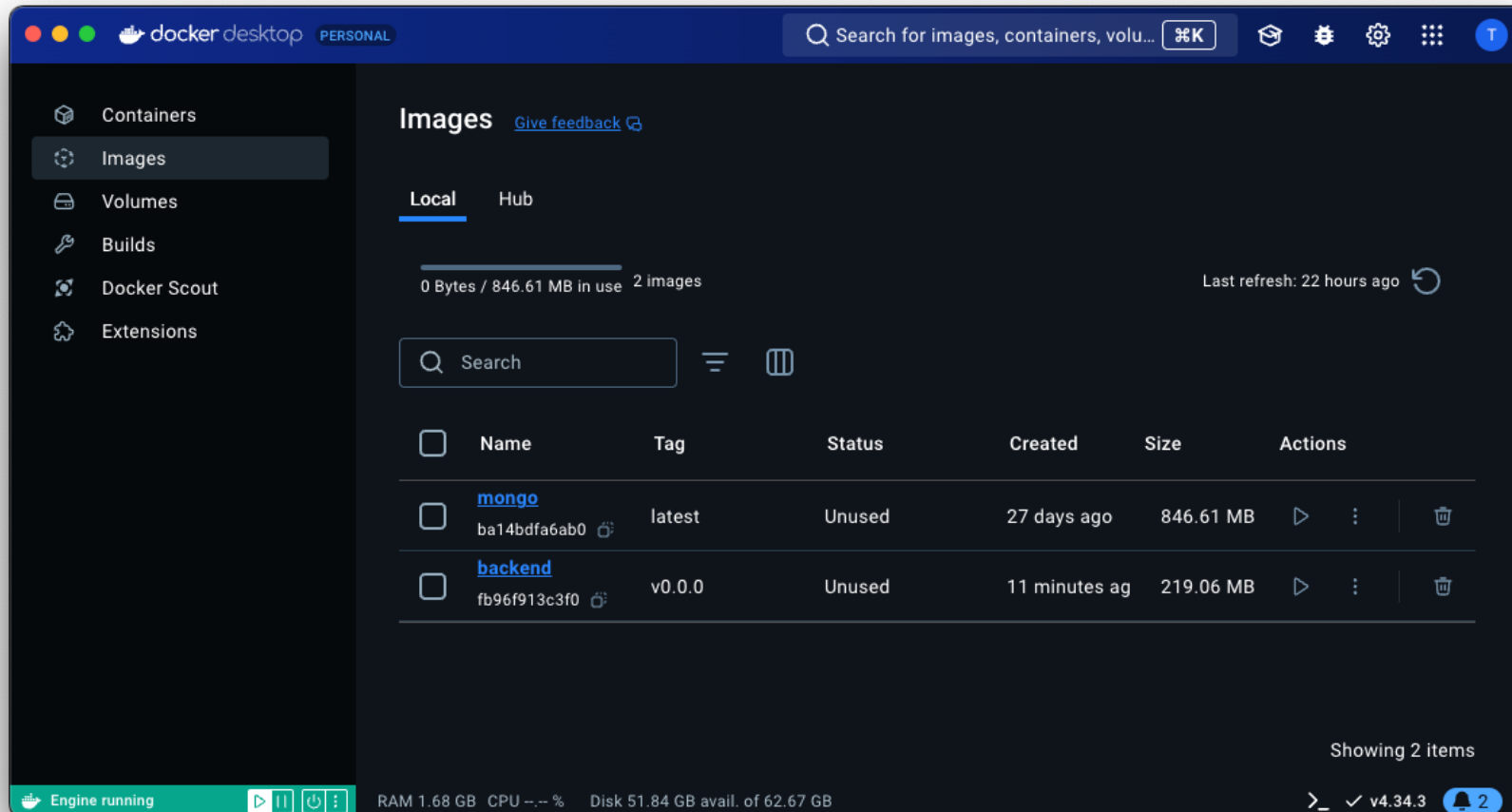
Docker Run

Docker Build

❖ Run container

```
docker run [OPTIONS] IMAGE [COMMAND] [ARG...]
```

```
docker run -it --name backend_cont backend:v0.0.0
```



Docker Build

❖ Exec to container

```
docker exec [OPTIONS] CONTAINER COMMAND [ARG...]
```

```
• thuannd@MacNaN backend % docker exec --help
```

```
Usage:  docker exec [OPTIONS] CONTAINER COMMAND [ARG...]
```

```
Execute a command in a running container
```

```
Aliases:
```

```
docker container exec, docker exec
```

```
Options:
```

-d, --detach	Detached mode: run command in the background
--detach-keys string	Override the key sequence for detaching a container
-e, --env list	Set environment variables
--env-file list	Read in a file of environment variables
-i, --interactive	Keep STDIN open even if not attached
--privileged	Give extended privileges to the command
-t, --tty	Allocate a pseudo-TTY
-u, --user string	Username or UID (format: "<name uid>[:<group gid>]")
-w, --workdir string	Working directory inside the container

Docker Build

❖ Exec to container

```
docker exec [OPTIONS] CONTAINER COMMAND [ARG...]
```

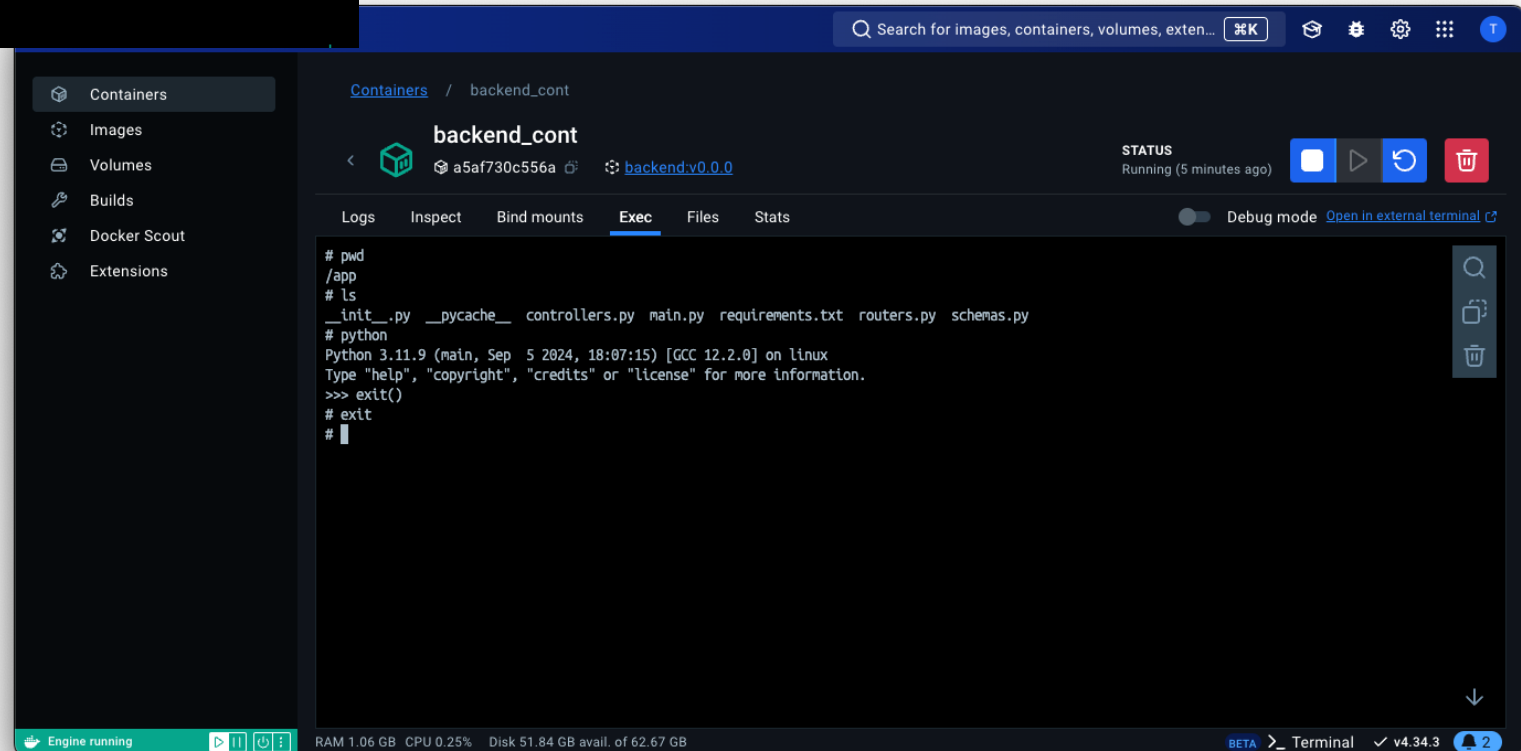
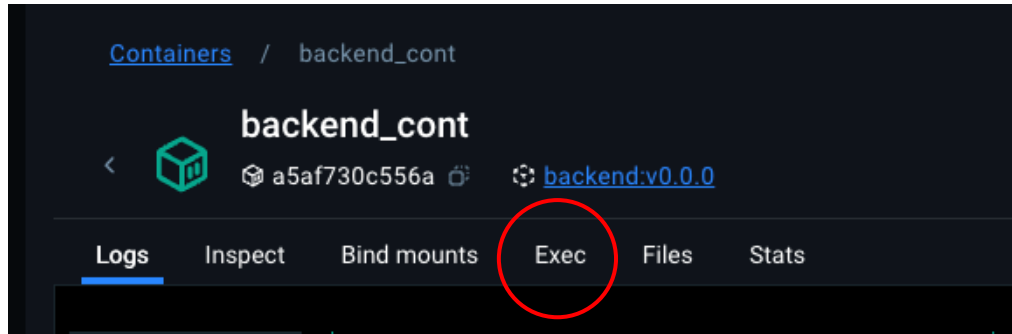
```
docker exec -it backend_cont bash
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS  DEBUG CONSOLE  JUPYTER

• thuannd@MacNaN backend % docker exec -it backend_cont bash
root@a5af730c556a:/app# pwd
/app
root@a5af730c556a:/app# ls
__init__.py  __pycache__  controllers.py  main.py  requirements.txt  routers.py  schemas.py
root@a5af730c556a:/app# python
Python 3.11.9 (main, Sep  5 2024, 18:07:15) [GCC 12.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> exit()
root@a5af730c556a:/app# exit
exit
```


Docker Build

❖ Exec to container



Docker Build

❖ Container logs

`docker logs [OPTIONS] CONTAINER`

`docker logs backend_cont`

```
thuannd@MacNaN backend % docker logs --help

Usage:  docker logs [OPTIONS] CONTAINER

Fetch the logs of a container

Aliases:
  docker container logs, docker logs

Options:
  --details      Show extra details provided to logs
  -f, --follow   Follow log output
  --since string Show logs since timestamp (e.g. "2013-01-02T13:23:37Z")
  -n, --tail string Number of lines to show from the end of the logs (default: 10)
  -t, --timestamps Show timestamps
  --until string Show logs before a timestamp (e.g. "2013-01-02T13:23:37Z")
```

```
thuannd@MacNaN backend % docker logs backend_cont
INFO    Using path main.py
INFO    Resolved absolute path /app/main.py
INFO    Searching for package file structure from directories with __init__.py files
INFO    Importing from /
```

Python package file structure

```
graph LR
    app[app] --> __init__[__init__.py]
    app --> main[main.py]
```

```
INFO    Importing module app.main
INFO    Found importable FastAPI app
```

Importable FastAPI app

```
from app.main import app
```

```
INFO    Using import string app.main:app
```

FastAPI CLI - Production mode

Serving at: <http://0.0.0.0:8000>

API docs: <http://0.0.0.0:8000/docs>

Running in production mode, for development use:

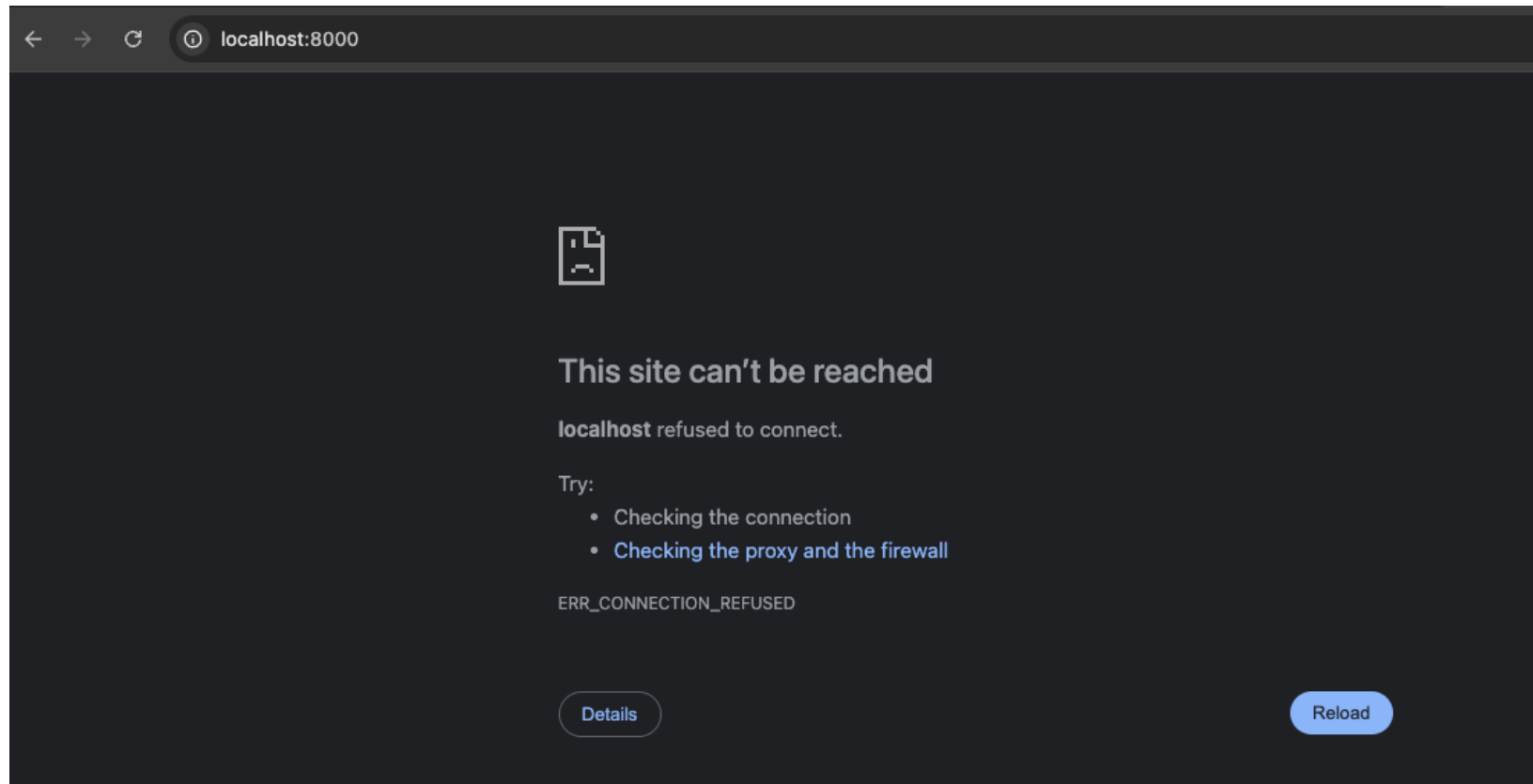
fastapi dev

```
INFO:    Started server process [1]
INFO:    Waiting for application startup.
INFO:    Application startup complete.
INFO:    Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
thuannd@MacNaN backend %
```

Docker Build

❖ Mapping Port

```
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```



Docker Build

❖ Mapping Port

```
docker run -it --name backend_cont -p 8000:8000 backend:v0.0.0
```

```
thuannd@MacNaN backend % docker run -it --name backend_cont -p 8000:8000 backend:v0.0.0
INFO    Using path main.py
INFO    Resolved absolute path /app/main.py
INFO    Searching for package file structure from directories with __init__.py files
INFO    Importing from /

Python package file structure
├── app
│   ├── __init__.py
│   └── main.py
└──

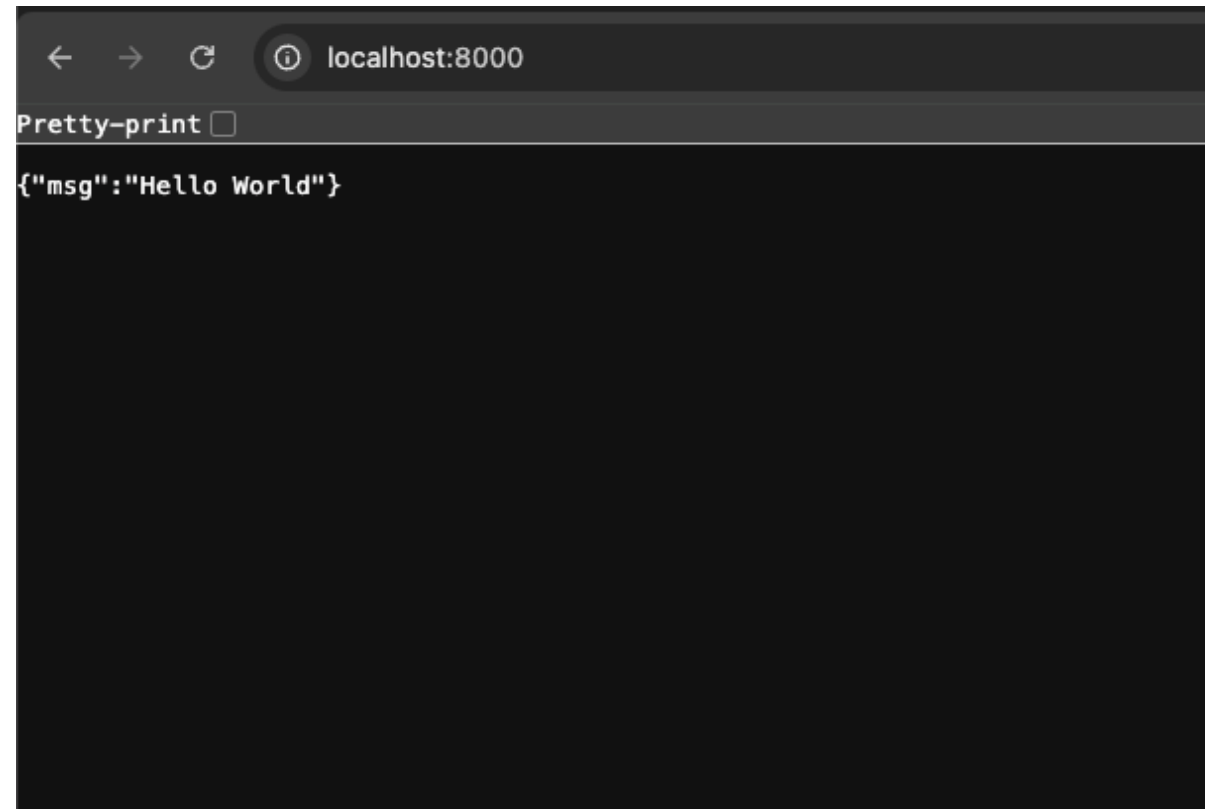
INFO    Importing module app.main
INFO    Found importable FastAPI app

Importable FastAPI app
from app.main import app

INFO    Using import string app.main:app

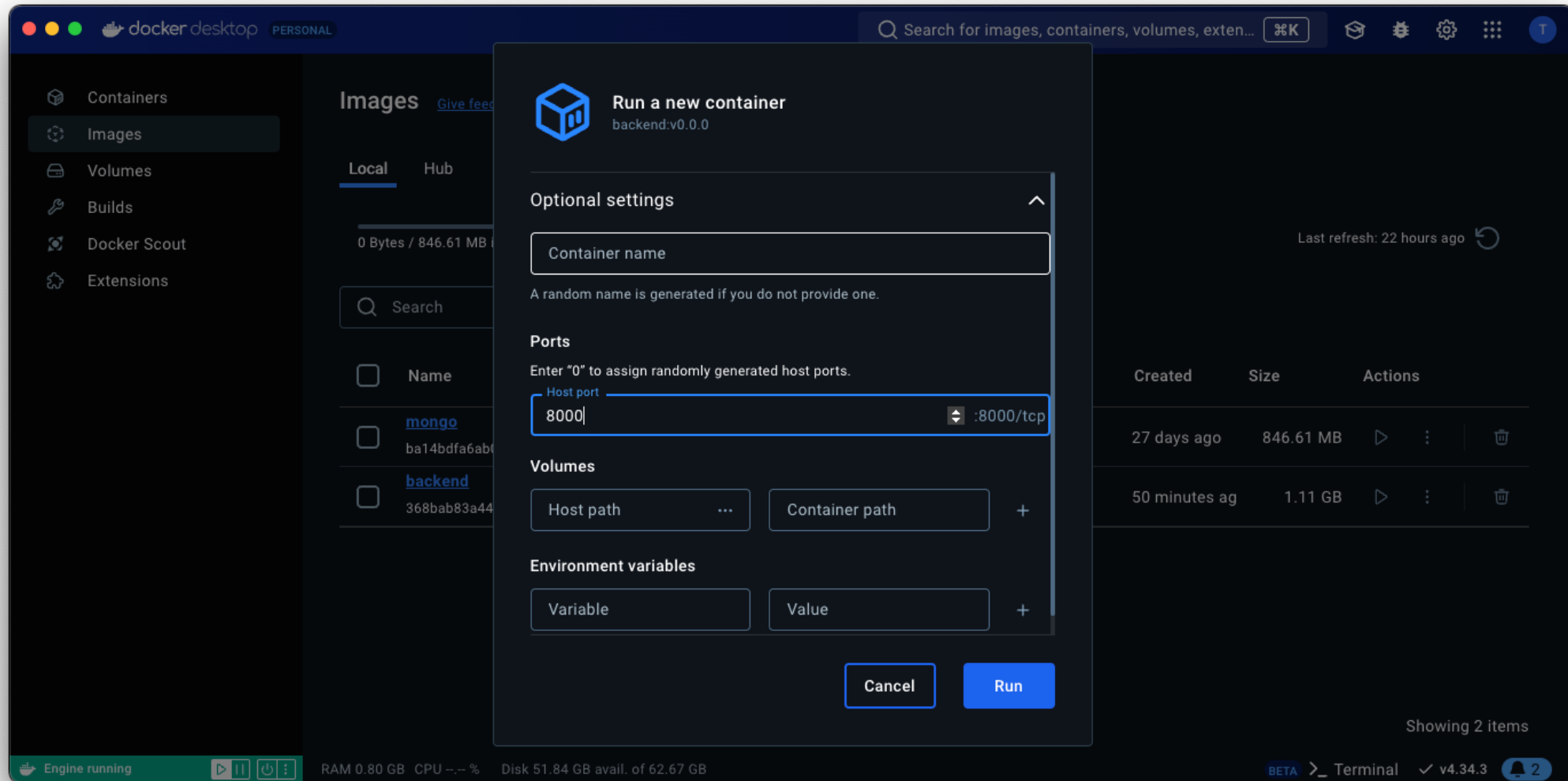
FastAPI CLI - Production mode
Serving at: http://0.0.0.0:8000
API docs: http://0.0.0.0:8000/docs
Running in production mode, for development use:
fastapi dev

INFO:    Started server process [1]
INFO:    Waiting for application startup.
INFO:    Application startup complete.
INFO:    Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```



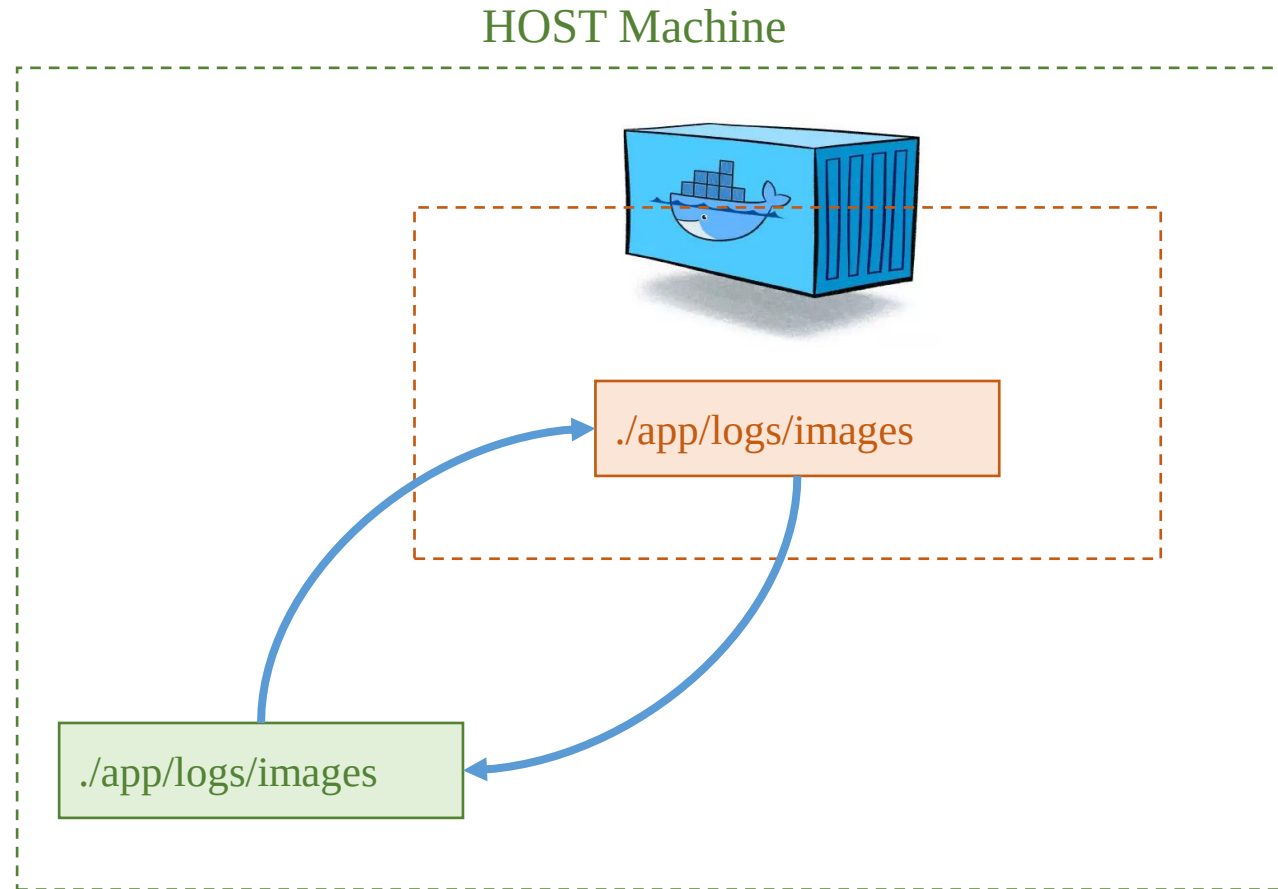
Docker Build

❖ Mapping Port



Docker Build

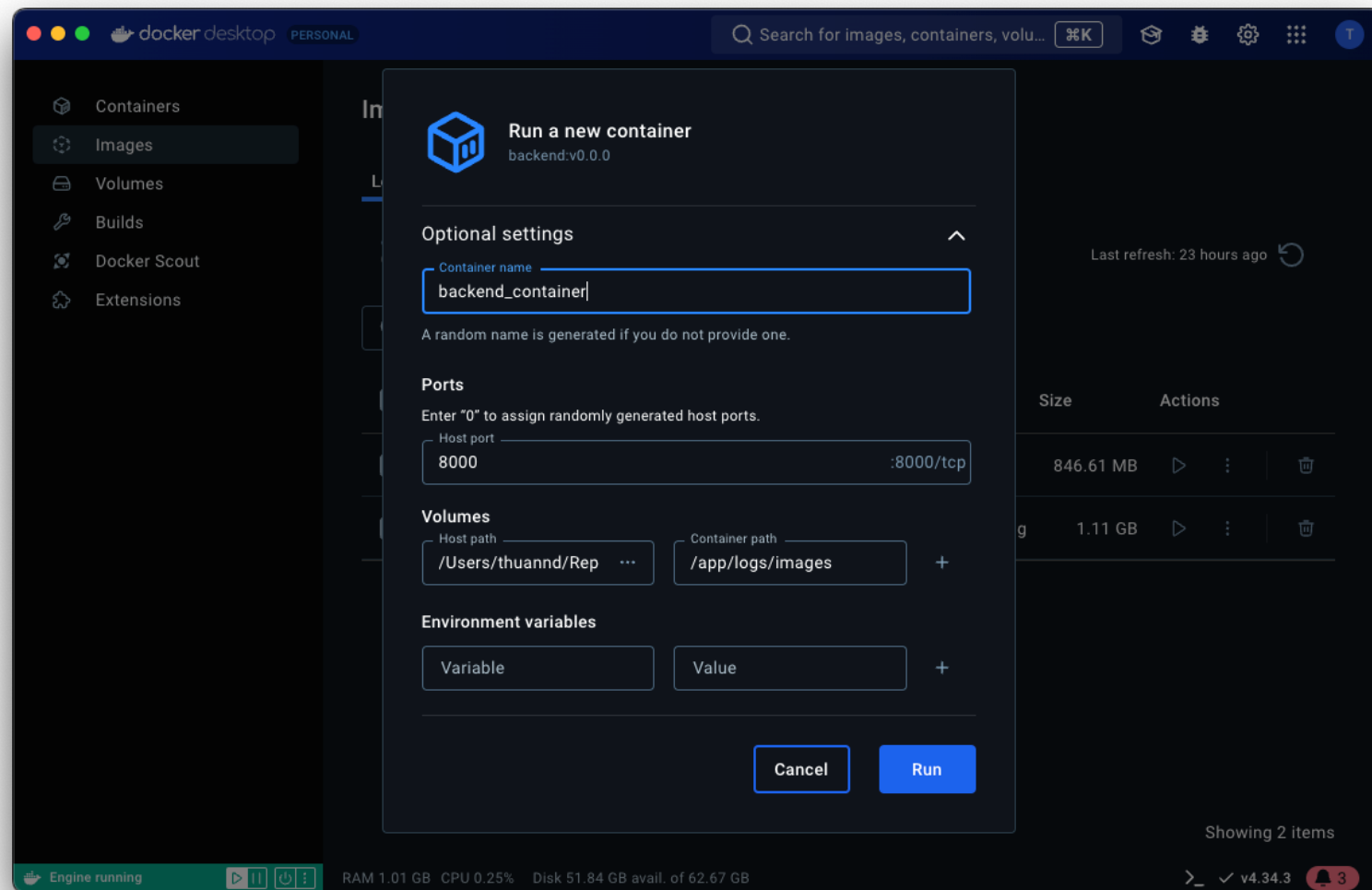
❖ Bind Mount Volume



Docker Build

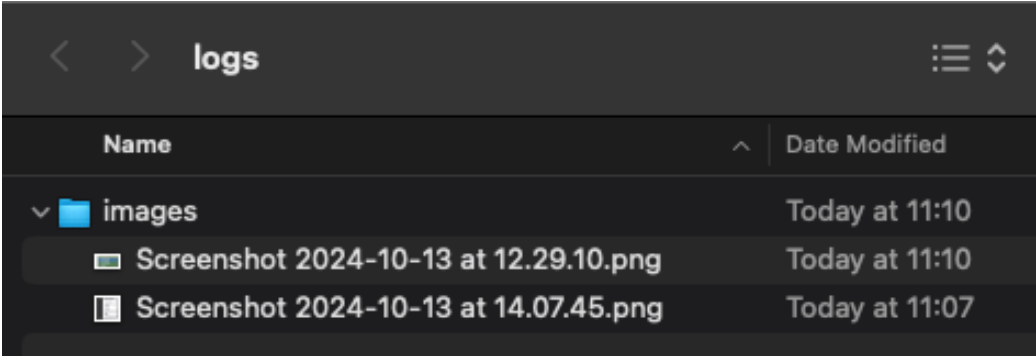
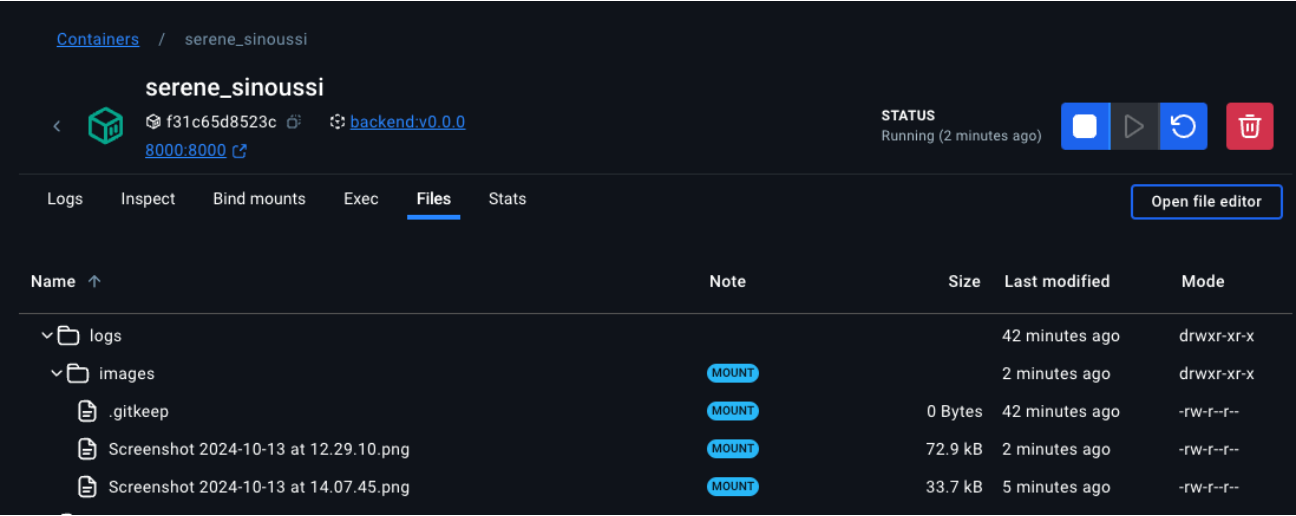
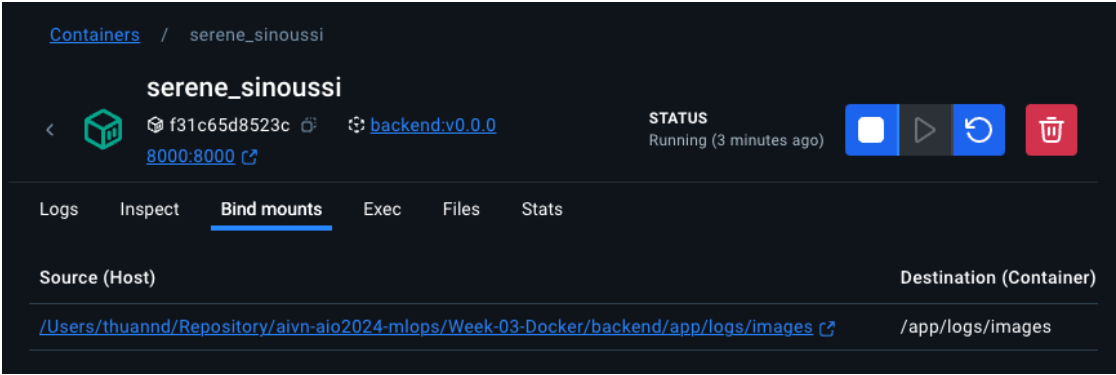
❖ Bind Mount Volume

```
docker run -it --name backend_cont -p 8000:8000 -v .app/logs/images:/app/logs/images backend:v0.0.0
```



Docker Build

❖ Bind Mount Volume



Docker Compose

Docker Compose

❖ Getting Started

- Docker Compose is a tool for defining and running multi-container applications.
- Allow control of your entire application stack, making it easy to manage services, networks, and volumes in a single comprehensible YAML configuration file.



Docker Compose

❖ Configuration file

docker-compose.yml

```
1  services:
2    backend:
3      build:
4        context: ./backend
5      container_name: backend_container
6      volumes:
7        - ./backend/app/logs/images:/app/logs/images
8      ports:
9        - 8000:8000
10     entrypoint: "uvicorn app.main:app --host 0.0.0.0 --port 8000"
11
12    frontend:
13      build:
14        context: ./frontend
15      container_name: frontend_container
16      ports:
17        - 3000:3000
18     entrypoint: "python app.py"
19
```

Docker Compose

❖ Run Compose

Usage: docker compose [OPTIONS] COMMAND

docker compose up -d

☐	Name	Image	Status	Port(s)
☐	 week-03-docker		Running (2/2)	
☐	 backend_container 13d14ae2b503 	week-03-docker-backend:<none>	Running	8000:8000 
☐	 frontend_container 781ef5f877a2 	week-03-docker-frontend:<none>	Running	3000:3000 

```
thuannd@MacNaN Week-03-Docker % docker compose up -d
[+] Building 6.1s (20/20) FINISHED
=> [frontend internal] load build definition from Dockerfile
=> => transferring dockerfile: 270B
=> [backend internal] load build definition from Dockerfile
=> => transferring dockerfile: 333B
=> [backend internal] load metadata for docker.io/library/python:3.11.9-slim
=> [backend internal] load .dockerignore
=> => transferring context: 2B
=> [frontend internal] load .dockerignore
=> => transferring context: 2B
=> [frontend 1/5] FROM docker.io/library/python:3.11.9-slim@sha256:8fb099199b9f2d70342674bd
=> [backend internal] load build context
=> => transferring context: 2.22kB
=> [frontend internal] load build context
=> => transferring context: 1.26kB
=> CACHED [frontend 2/5] WORKDIR /app
=> CACHED [frontend 3/5] COPY ./requirements.txt /app/requirements.txt
=> CACHED [frontend 4/5] RUN pip install --no-cache-dir --upgrade -r /app/requirements.txt
=> CACHED [frontend 5/5] COPY ./app /app
=> CACHED [backend 2/5] WORKDIR /src
=> CACHED [backend 3/5] COPY ./requirements.txt /src/requirements.txt
=> CACHED [backend 4/5] RUN pip install --no-cache-dir --upgrade -r /src/requirements.txt
=> CACHED [backend 5/5] COPY ./app /src/app
=> [frontend] exporting to image
=> => exporting layers
=> => writing image sha256:1da40ef78601761ebaa94959c0b03f17cda3e7ece39becfdee365cd3f555adb5
=> => naming to docker.io/library/week-03-docker-frontend
=> [backend] exporting to image
=> => exporting layers
=> => writing image sha256:328b892c967db1ec5c6fdb40535396d076948c446944251797752c32e3f62df6
=> => naming to docker.io/library/week-03-docker-backend
=> [frontend] resolving provenance for metadata file
=> [backend] resolving provenance for metadata file
[+] Running 3/3
✓ Network week-03-docker_default Created
✓ Container backend_container Started
✓ Container frontend_container Started
```

Docker Compose

❖ Stop Compose

Usage: `docker compose [OPTIONS] COMMAND`

`docker compose down`

```
• thuannd@MacNaN Week-03-Docker % docker compose down
[+] Running 3/3
  ✓ Container backend_container      Removed
  ✓ Container frontend_container     Removed
  ✓ Network week-03-docker_default   Removed
○ thuannd@MacNaN Week-03-Docker %
```

Containers [Give feedback](#)



Your running containers show up here

A container is an isolated environment for your code

	What is a container? 5 mins		How do I run a container? 6 mins
---	---------------------------------------	---	--

[View more in the Learning center](#)

Docker Compose

❖ Docker Compose Network

The image shows a Docker Desktop interface. On the left, there is a web application titled "OCR" running on localhost:3000. It displays a list of topics: Introduction, Docker Desktop, Docker Engine, Docker Build, Docker Compose, Docker Hub, and Question. Below the list are "Clear" and "Submit" buttons. On the right, the Docker Desktop interface shows the "frontend_container" running. The container's status is "Running (2 minutes ago)". The logs for the container show a series of Python requests and a final error message: "requests.exceptions.ConnectionError: HTTPConnectionPool(host='0.0.0.0', port=8000): Max retries exceeded with url: /ocr/predict (Caused by NewConnectionError('<urllib3.connection.HTTPConnection object at 0xffff757d58d0>: Failed to establish a new connection: [Errno 111] Connection refused'))".

OCR

This is a OCR application

Input Image

- Introduction
- Docker Desktop
- Docker Engine
- Docker Build
- Docker Compose
- Docker Hub
- Question

Clear Submit

Status

Error

Output Image

docker desktop PERSONAL

Containers / frontend_container

frontend_container

781ef5f877a2 (was week-03-docker-frontend-<none>) 3000:3000

STATUS Running (2 minutes ago)

Logs

```
2024-10-16 19:27:47 File "/app/api/ocr.py", line 20, in ocr_api
2024-10-16 19:27:47     response = requests.post(url, headers=headers, files=files)
2024-10-16 19:27:47 ~~~~~
2024-10-16 19:27:47 File "/usr/local/lib/python3.11/site-packages/requests/api.py", line 115, in post
2024-10-16 19:27:47     return request("post", url, data=data, json=json, **kwargs)
2024-10-16 19:27:47 ~~~~~
2024-10-16 19:27:47 File "/usr/local/lib/python3.11/site-packages/requests/api.py", line 59, in request
2024-10-16 19:27:47     return session.request(method=method, url=url, **kwargs)
2024-10-16 19:27:47 ~~~~~
2024-10-16 19:27:47 File "/usr/local/lib/python3.11/site-packages/requests/sessions.py", line 589, in request
2024-10-16 19:27:47     resp = self.send(prepare, **send_kwargs)
2024-10-16 19:27:47 ~~~~~
2024-10-16 19:27:47 File "/usr/local/lib/python3.11/site-packages/requests/sessions.py", line 783, in send
2024-10-16 19:27:47     r = adapter.send(request, **kwargs)
2024-10-16 19:27:47 ~~~~~
2024-10-16 19:27:47 File "/usr/local/lib/python3.11/site-packages/requests/adapters.py", line 700, in send
2024-10-16 19:27:47     raise ConnectionError(e, request=request)
2024-10-16 19:27:47 requests.exceptions.ConnectionError: HTTPConnectionPool(host='0.0.0.0', port=8000): Max retries exceeded with url: /ocr/predict (Caused by NewConnectionError('<urllib3.connection.HTTPConnection object at 0xffff757d58d0>: Failed to establish a new connection: [Errno 111] Connection refused'))
```

RAM 3.99 GB CPU 0.25% Disk 49.36 GB avail. of 62.67 GB

BETA Terminal v4.34.3 3

Docker Compose

❖ Docker Compose Network

Config network in docker-compose.yml file

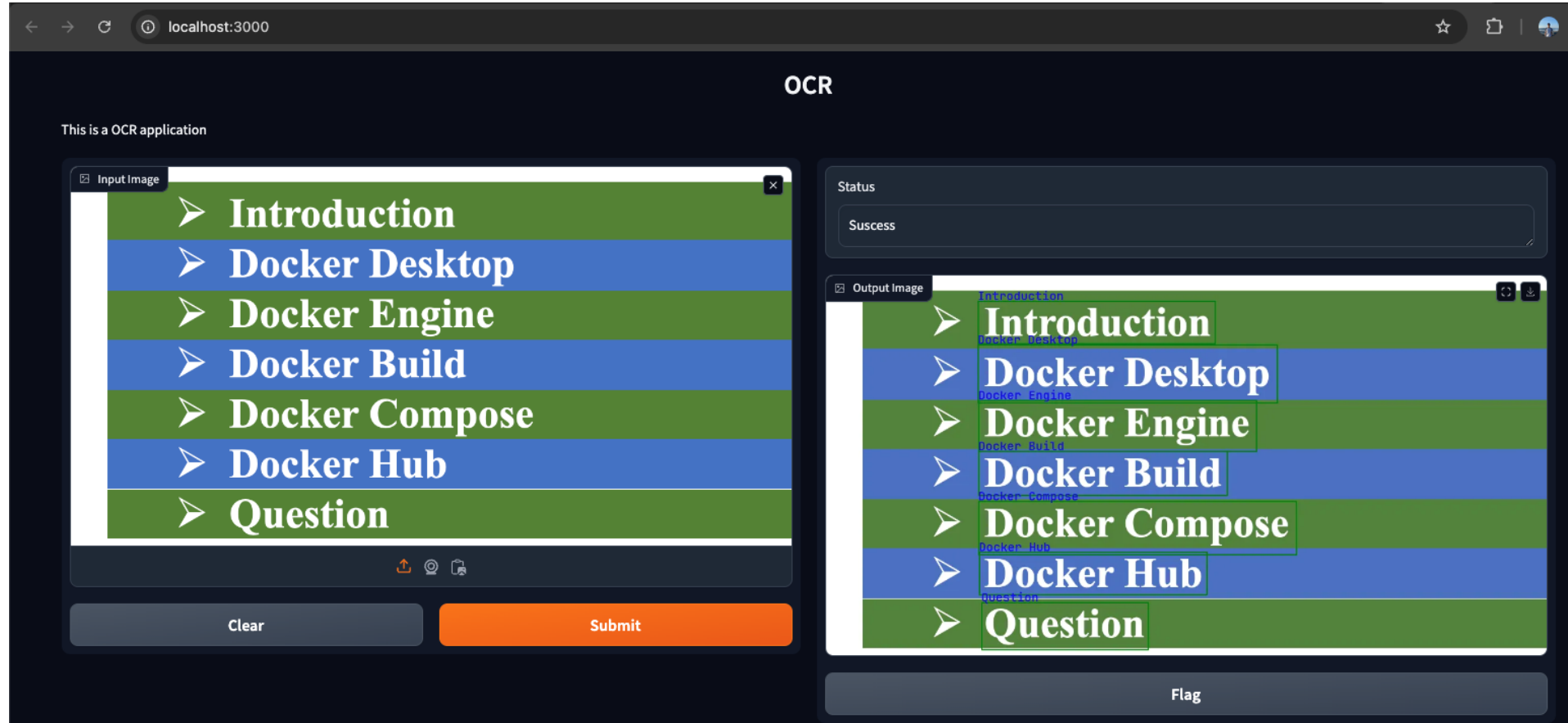
```
docker compose up -d
```

```
=> [backend] resolving provenance for metadata file
[+] Running 3/3
✓ Network week-03-docker_aio_mlops_net Created
✓ Container backend_cont Started
✓ Container frontend_cont Started
```

```
1  services:
2    backend:
3      build:
4        context: ./backend
5        container_name: backend_cont
6      volumes:
7        - ./backend/app/logs/images:/app/logs/images
8      networks:
9        - aio_mlops_net
10     ports:
11       - 8000:8000
12     entrypoint: "uvicorn app.main:app --host 0.0.0.0 --port 8000"
13
14     frontend:
15       build:
16         context: ./frontend
17         container_name: frontend_cont
18       env_file:
19         - ./frontend/.env
20       networks:
21         - aio_mlops_net
22       ports:
23         - 3000:3000
24       entrypoint: "python app.py"
25
26     networks:
27       aio_mlops_net:
28         driver: bridge
```

Docker Compose

❖ Docker Compose Network

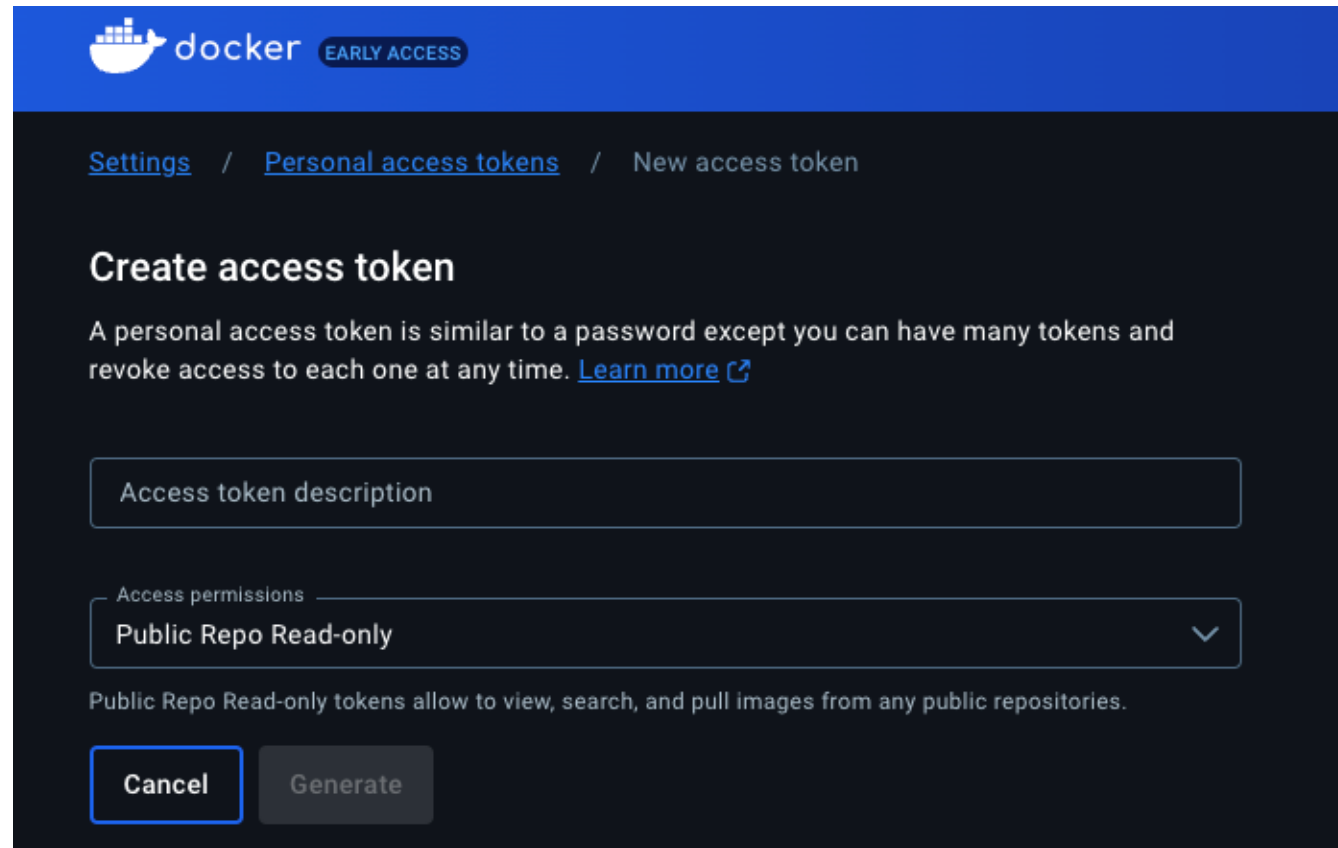


Docker Hub

Docker Hub

❖ Getting Started

- Login by token at: <https://app.docker.com/settings/personal-access-tokens>
- Tagging your local image with Docker Hub username.
- Push image to Hub



The screenshot shows the Docker Hub interface for creating a new access token. At the top, there's a blue header with the Docker logo and 'EARLY ACCESS' badge. Below the header, a breadcrumb trail reads 'Settings / Personal access tokens / New access token'. The main heading is 'Create access token'. A descriptive paragraph explains that a personal access token is similar to a password but can be revoked and that multiple tokens can be created, with a 'Learn more' link. Below this is a form with two main sections: 'Access token description' with a text input field, and 'Access permissions' with a dropdown menu currently set to 'Public Repo Read-only'. A note below the dropdown states: 'Public Repo Read-only tokens allow to view, search, and pull images from any public repositories.' At the bottom, there are two buttons: 'Cancel' and 'Generate'.

docker EARLY ACCESS

[Settings](#) / [Personal access tokens](#) / New access token

Create access token

A personal access token is similar to a password except you can have many tokens and revoke access to each one at any time. [Learn more](#)

Access token description

Access permissions

Public Repo Read-only

Public Repo Read-only tokens allow to view, search, and pull images from any public repositories.

Cancel Generate

Docker Hub

❖ Tagging

- Login by token at: <https://app.docker.com/settings/personal-access-tokens>
- Tagging your local image with Docker Hub username.
- Push image to Hub

```
docker tag week-03-docker-frontend thuannan/week-03-docker-frontend
```

☐	Name	Tag
☐	week-03-docker-backend b0b389a63b71 	latest
☐	week-03-docker-frontend 1da40ef78601 	latest
☐	thuannan/week-03-docker-frontend 1da40ef78601 	latest
☐	mongo ba14bdfa6ab0 	latest

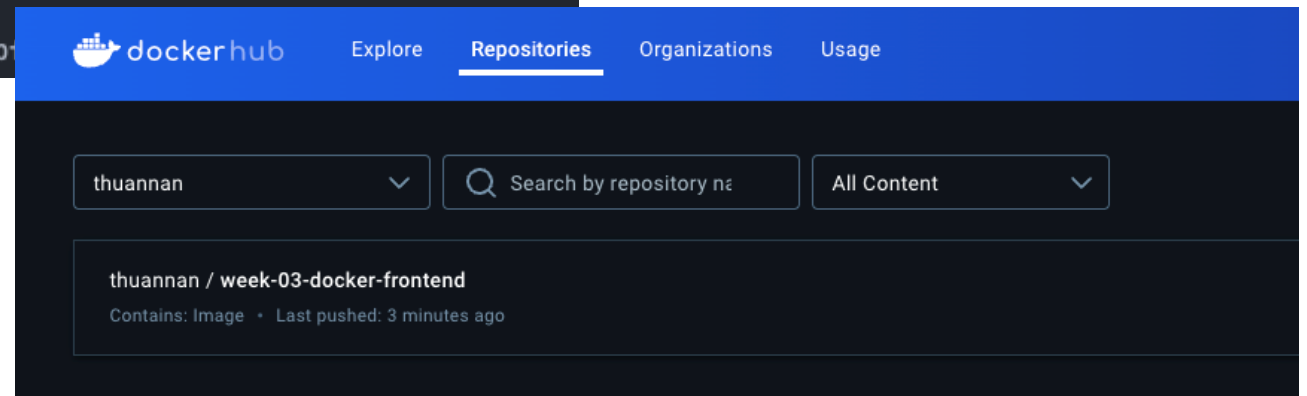
Docker Hub

❖ Pushing

- Login by token at: <https://app.docker.com/settings/personal-access-tokens>
- Tagging your local image with Docker Hub username.
- Push image to Hub

```
docker push thuannan/week-03-docker-frontend
```

```
(aio-mlops-w2) thuannan@MacNaN Week-03-Docker % docker push thuannan/week-03-docker-frontend
Using default tag: latest
The push refers to repository [docker.io/thuannan/week-03-docker-frontend]
0723cc98afde: Pushed
5aa879153ec6: Pushed
c5505ccb471b: Pushed
5c143465292d: Pushed
74c21085cca9: Pushed
84e14a567d37: Pushed
9137ed4faf20: Pushed
ad0a49486599: Pushed
e644ff0c302d: Pushed
latest: digest: sha256:db9ec205d71e38b242d92dd37957dae9c9c85855b...
```



Docker Hub

❖ Pulling

```
docker push thuannan/week-03-docker-frontend
```

```
thuannnd — thuannnd@ubuntu-nan0: ~ — ssh UbuntuHome0 — 105x24
[thuannnd@ubuntu-nan0:~$ docker pull thuannan/week-03-docker-frontend
Using default tag: latest
latest: Pulling from thuannan/week-03-docker-frontend
92c3b3500be6: Pull complete
3bc0bb0f4836: Pull complete
e1823f75652f: Pull complete
8fe47198239e: Pull complete
fd14ce64cb6c: Pull complete
62034efb7919: Pull complete
3015613b7d32: Pull complete
b596a0606163: Pull complete
ab5de96361eb: Pull complete
Digest: sha256:db9ec205d71e38b242d92dd37957dae9c9c85855bffc269e889a447472157da7
Status: Downloaded newer image for thuannan/week-03-docker-frontend:latest
docker.io/thuannan/week-03-docker-frontend:latest
[thuannnd@ubuntu-nan0:~$ docker images
REPOSITORY                TAG                IMAGE ID            CREATED             SIZE
thuannan/week-03-docker-frontend  latest            1da40ef78601        2 hours ago        534MB
hadoop-build-1000          latest            a409ff4a079c        2 days ago         2.41GB
hadoop-build               latest            62d70598ee4d        2 days ago         2.41GB
mongo                     latest            72576a3db032        3 weeks ago        855MB
thuannnd@ubuntu-nan0:~$
```

Summary

Summary

❖ Performance

The screenshot shows the Docker Desktop interface. The top bar includes the Docker logo, 'docker desktop PERSONAL', a search bar, and various icons. The left sidebar lists navigation options: Containers, Images, Volumes, Builds, Docker Scout, and Extensions. The main area is titled 'Containers' and displays performance metrics: 'Container CPU usage 0.48% / 800% (8 CPUs available)' and 'Container memory usage 536.1MB / 7.48GB'. Below these metrics is a search bar and a toggle for 'Only show running containers'. A table lists three running containers: 'week-03-docker', 'frontend_cont', and 'backend_cont'. The bottom status bar shows 'Engine running', system resources (RAM 4.51 GB, CPU 0.25%, Disk 49.25 GB avail. of 62.67 GB), and version information (v4.34.3).

	Name	Image	Status	Port(s)	CPU (%)	Last started	Actions
☐	week-03-docker		Running (2/2)		0.48%	3 minutes ago	
☐	frontend_cont f82ad6d19275	week-03-docker-frontend:<none>	Running	3000:3000	0.38%	3 minutes ago	
☐	backend_cont 94f65fc26e83	week-03-docker-backend:<none>	Running	8000:8000	0.1%	3 minutes ago	

Showing 3 items

Summary

❖ Docker CLI: image

Command	Function
<code>docker images</code>	To list all the Docker images on your system
<code>docker pull <image_name></code>	To download a Docker image from registry
<code>docker build -t <image_name>:<tag> <path_to_dockerfile></code>	To create a new image from a Dockerfile
<code>docker rmi <image_name_or_id></code>	To remove a Docker image
<code>docker tag <existing_image> <new_image_name>:<tag></code>	To tag an image with a new name
<code>docker push <image_name>:<tag></code>	To push an image to a Docker registry

Summary

❖ Docker CLI: container

Command	Function
<code>docker ps</code>	To list all running containers
<code>docker run <image_name></code>	To create and run a new container from an image
<code>docker stop <container_id_or_name></code>	To stop a running container
<code>docker start <container_id_or_name></code>	To start a container that was previously stopped
<code>docker rm <container_id_or_name></code>	To delete a container
<code>docker logs <container_id_or_name></code>	To see the logs of a container
<code>docker exec -it <container_id_or_name> <command></code>	To run a command inside an already running container

Question

